

E. Generative Modeling

E.1 Generative Modeling: Basic concepts

E.2 Generative Modeling: Variational models

E.3 Generative Modeling: Diffusion models I

E.4 Generative Modeling: Diffusion models II

Diffusion models, as reverse SDEs

E.5 Generative Modeling: Structured (conditional) generation

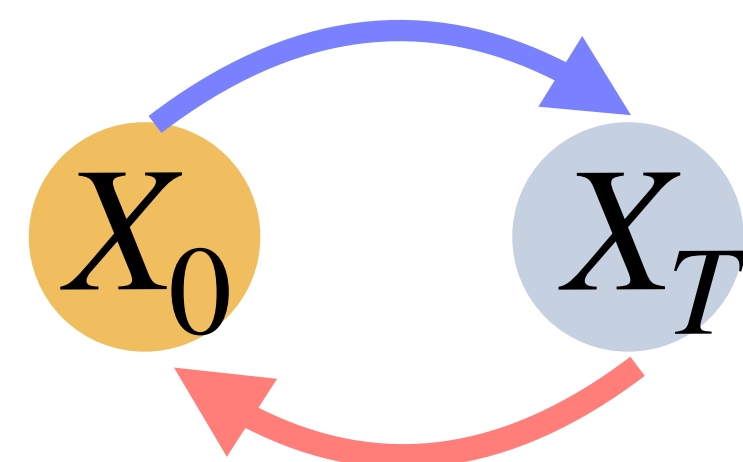


Tiam Heydari, PhD
Postdoctoral Fellow, UBC

Introduction: What we will learn in this lecture

Diffusion Models train a neural network to undo the noise added to each data point

Forward: add noise to data over T steps



Backward: start from noise and denoise in T steps

Forward: add noise to data over T steps



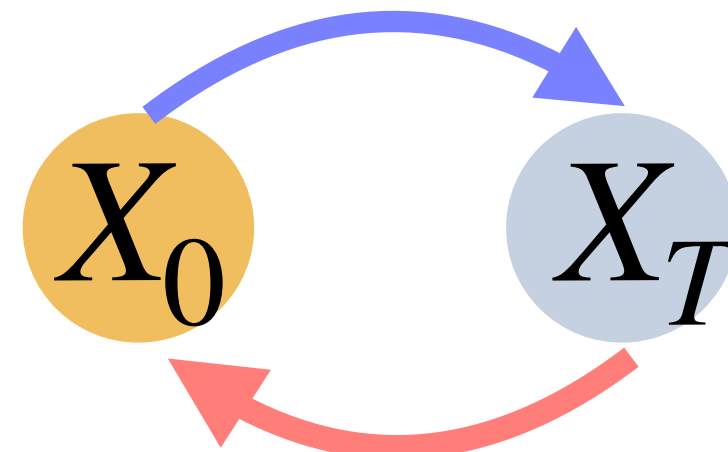
Backward: start from noise and denoise in T steps



Introduction: Diffusion as SDEs

This process is basically an stochastic differential equation problem and we have introduced the machinery to forward SDEs and reverse SDEs.

Forward: add noise to data over T steps



Backward: start from noise and denoise in T steps

Forward SDE:

Initial condition: data (x_0)

End point: Gaussian distribution (X_T)

- i) We need to know the force field $F(x, t)$
- ii) We need to know or define noise scale $\sigma(t)$
- iii) Forward SDE is:

$$dx = F(x, t) dt + \sigma(t) dW_t$$

$$dW_t = \sqrt{dt} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

reverse an SDE

Initial condition: Gaussian distribution (X_T)

End point: data (x_0)

- i) flip the force: $F(x, t) \rightarrow -F(x, t)$
- ii) We need to learn $\nabla_x \log p_t(x)$ (unknown) where $p_t(x)$ is density of the particle/data cloud at time t .

iii) Reverse SDE is:

$$dx = \left[-F(x, t) + \sigma(t)^2 \nabla_x \log p_t(x) \right] dt + \sigma(t) d\bar{W}_t$$

1) Forward SDE process : Letting data points diffuse freely

Let's start with the simplest forward SDE: no force, only noise:

$$dx = \sigma dW_t$$

Euler-Maruyama:

$$x_{t+\Delta t} = x_t + \sigma \sqrt{\Delta t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

$\Delta t = 1$ gives: $x_{t+1} = x_t + \sigma \epsilon$

Problem1: Pure diffusion destroys structure, but the cloud keeps expanding

Why this happens? At each step: $x_{t+1} = x_t + \sigma \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$

The variance of x_{t+1} is*: $= \text{Var}(x_t) + \text{Var}(\sigma \epsilon)$

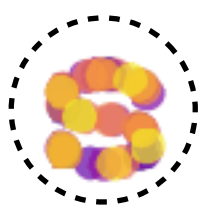
$$\text{Var}(x_{t+1}) = \text{Var}(x_t + \sigma \epsilon) = \text{Var}(x_t) + \sigma^2$$

In free diffusion variance keep increasing!

* $\text{Var}(ax + by) = a^2 \text{Var}(x) + b^2 \text{Var}(y) \quad ; \text{if} : x \perp y$

Reminder: **Forward SDE:**
$$dx = F(x, t) dt + \sigma(t) dW_t$$
$$dW_t = \sqrt{dt} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

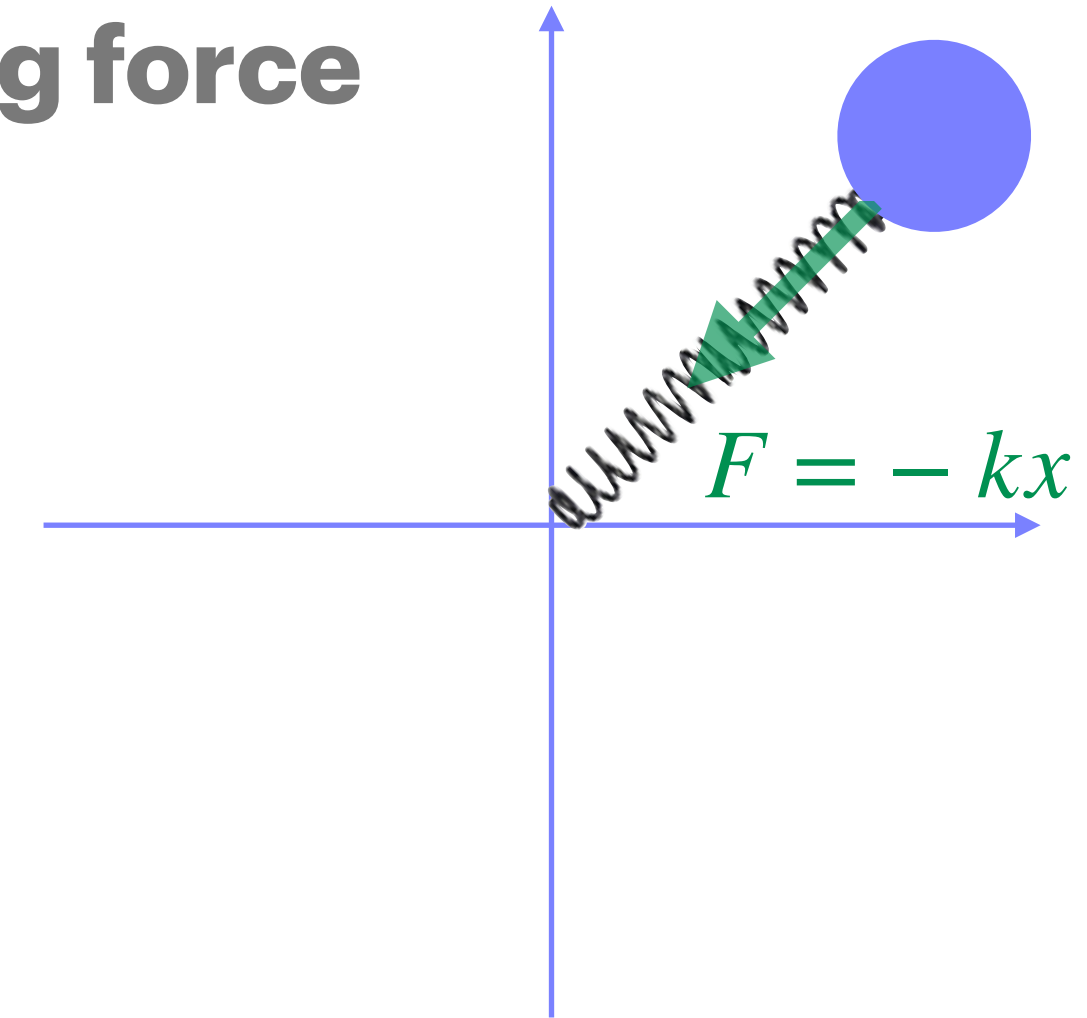
Reminder: Euler-Maruyama:
$$x_{n+1} = x_n + F(x_n, t_n) \Delta t + \sigma(t_n) \sqrt{\Delta t} \epsilon$$
$$\epsilon \sim \mathcal{N}(0, I_d)$$



1) Forward SDE process: Letting data points diffuse but add a restoring force

Solution: The idea is:

As we saw, when points are diffusion freely, they spread.
To solve the problem of spreading, we can add a force.
We choose the simplest restoring force: a spring force toward the origin.



$$F = -kx_t$$

What should k be to to keep Variance=1?

$$dx = -kx dt + \sigma dW_t \quad \xrightarrow{\text{discret form}}$$

$$x_{t+\Delta t} = x_t - kx_t\Delta t + \sigma\sqrt{\Delta t}\epsilon = (1 - k\Delta t)x_t + \sigma\sqrt{\Delta t}\epsilon \quad \xrightarrow{\text{Calculate Var}}$$

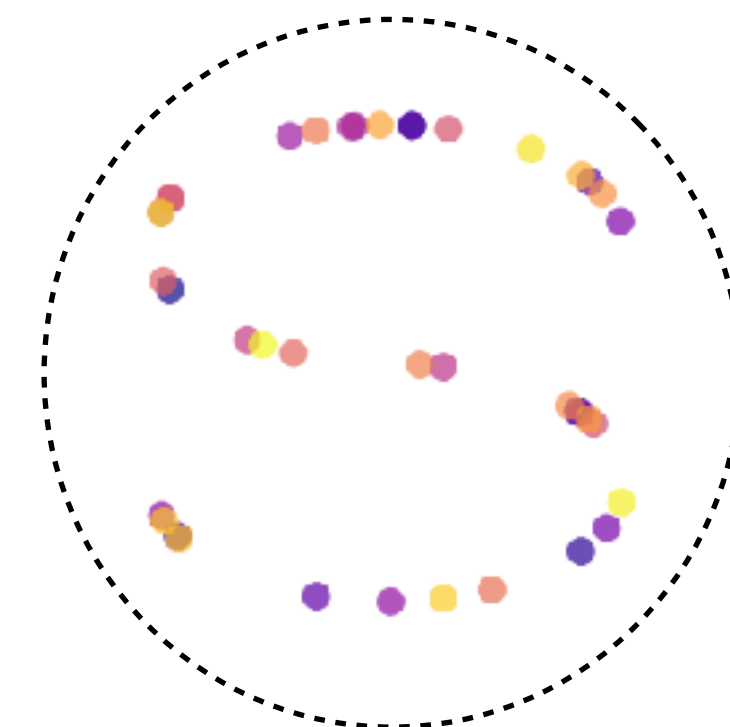
$$\text{Var}(x_{t+\Delta t}) = (1 - k\Delta t)^2\text{Var}(x_t) + \sigma^2\Delta t \approx (1 - k\Delta t)^2 + \sigma^2\Delta t \quad \xrightarrow{\text{Taylor expansion: } (1-k\Delta t)^2 \approx 1-2k\Delta t}$$

$$\text{Var}(x_{t+\Delta t}) \approx 1 - 2k\Delta t + \sigma^2\Delta t \stackrel{\text{we assert}}{=} 1 \quad \xrightarrow{\text{Simplify}}$$

$$k = \frac{1}{2}\sigma^2 \quad \xrightarrow{\text{Define } \beta}$$

$$\sigma^2 = \beta \quad \xrightarrow{\text{The equations become}}$$

$$dx = -\frac{1}{2}\beta x dt + \sqrt{\beta(t)} dW_t \quad \text{or discrete: } x_{t+\Delta t} = x_t - \frac{1}{2}\beta x_t\Delta t + \sqrt{\beta(t)}\sqrt{\Delta t}\epsilon, \epsilon \sim \mathcal{N}(0,I)$$



1) Forward SDE process, the final form: a controlled path from data to noise

Summary of forward SDE:

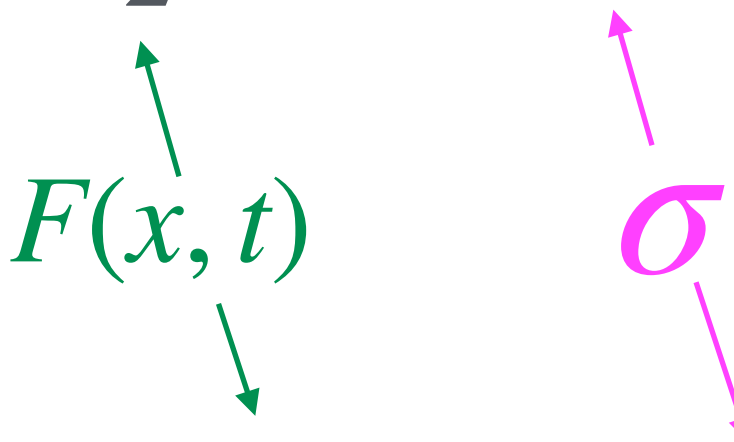
We start from clean data: $x_0 \sim p_{\text{data}}$

and define a fixed noising process (trajectories): $x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_T$

β = noise coefficient

Continuous version:

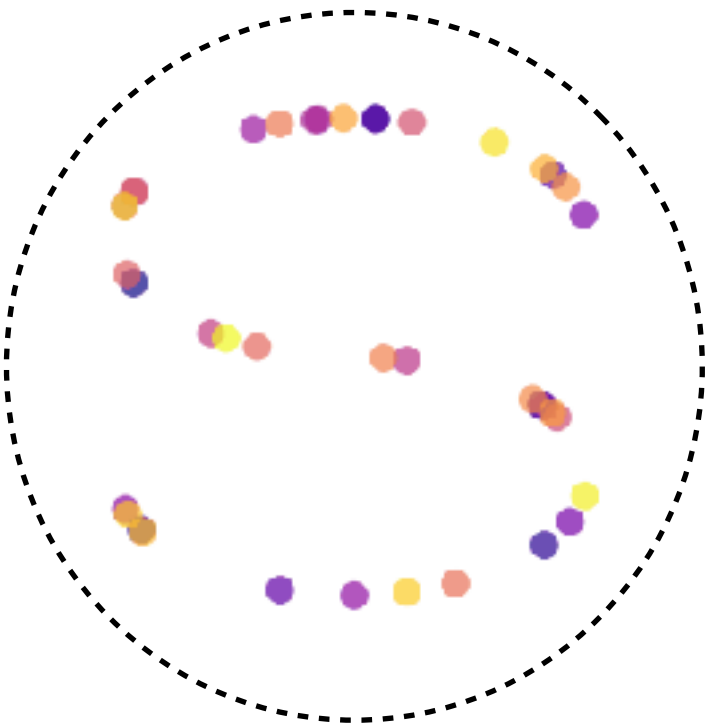
$$dx = \left(-\frac{1}{2}\beta x\right) dt + \sqrt{\beta} dW_t \quad ;\text{where: } dW_t = \sqrt{dt} \epsilon \text{ and } \epsilon \sim \mathcal{N}(0,1)$$



Discrete version:

$$x_{t+\Delta t} - x_t = \left(-\frac{1}{2}\beta x_t\right) \Delta t + \sqrt{\beta} \sqrt{\Delta t} \epsilon \quad ;\text{where: } \epsilon_t \sim \mathcal{N}(0,I)$$

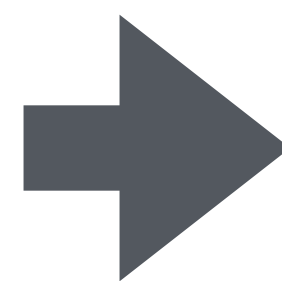
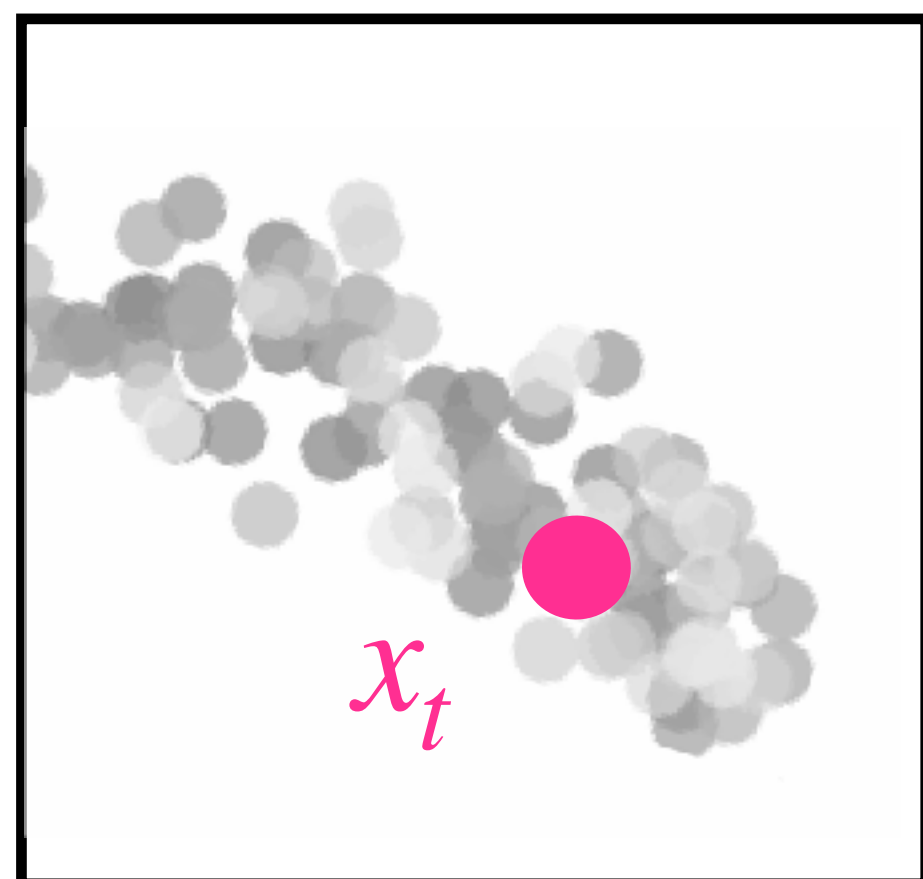
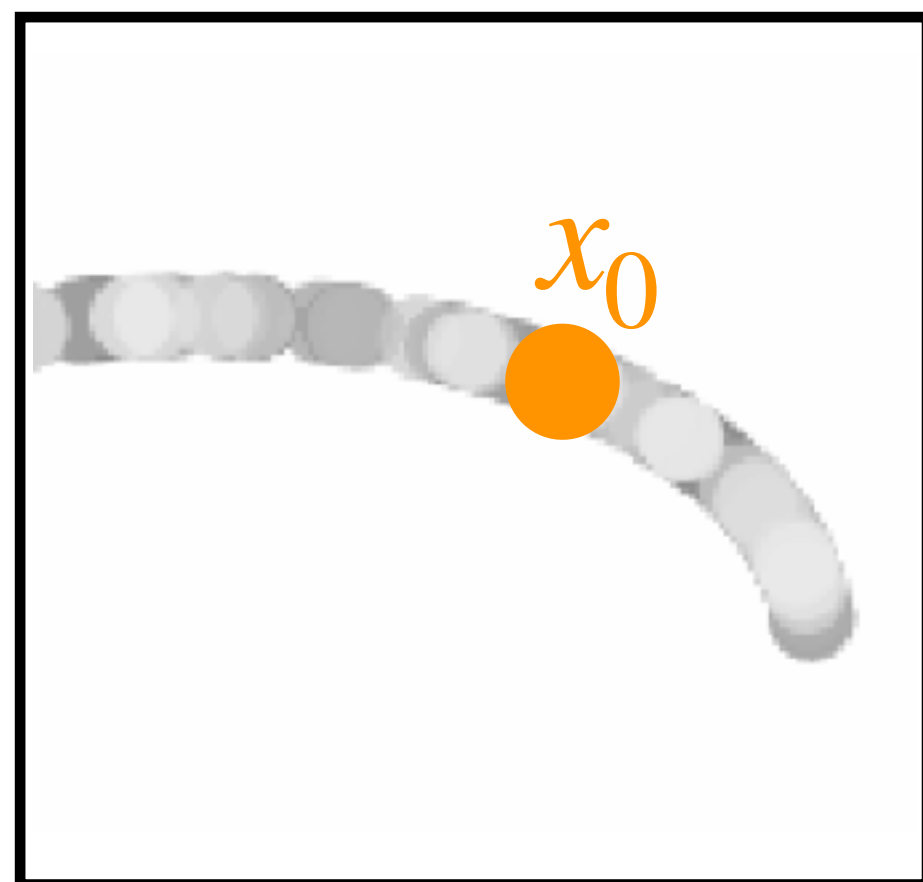
we can simulate the forward process with Euler-Maruyama:



Clarifying our probability terminology

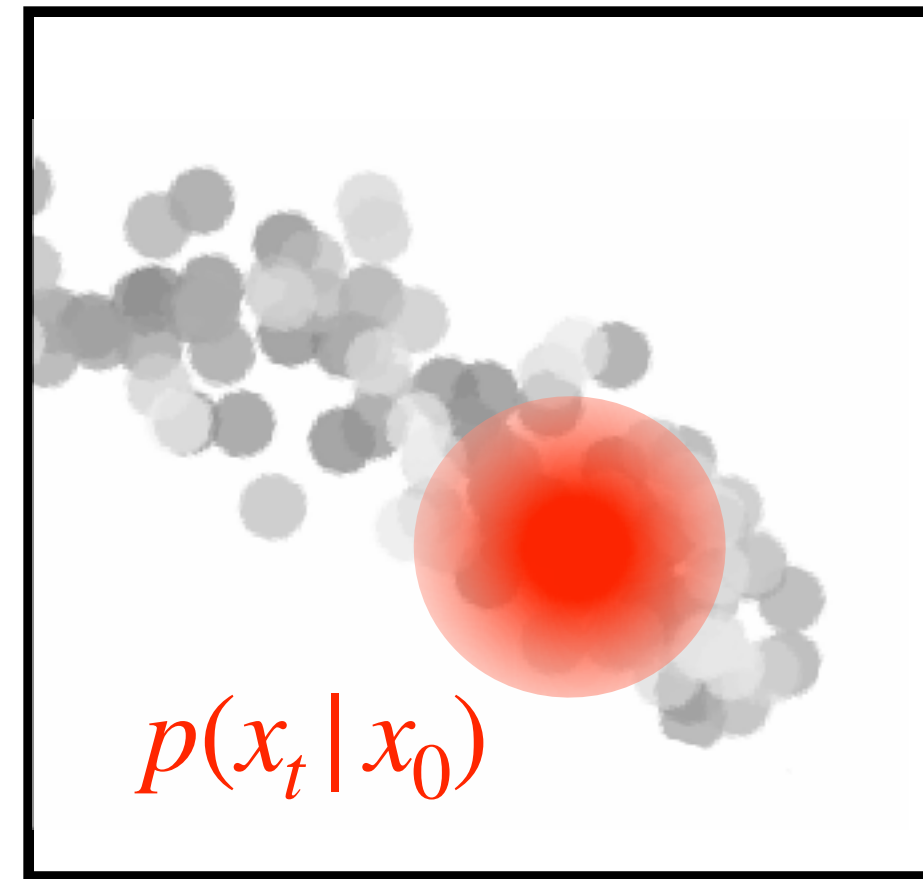
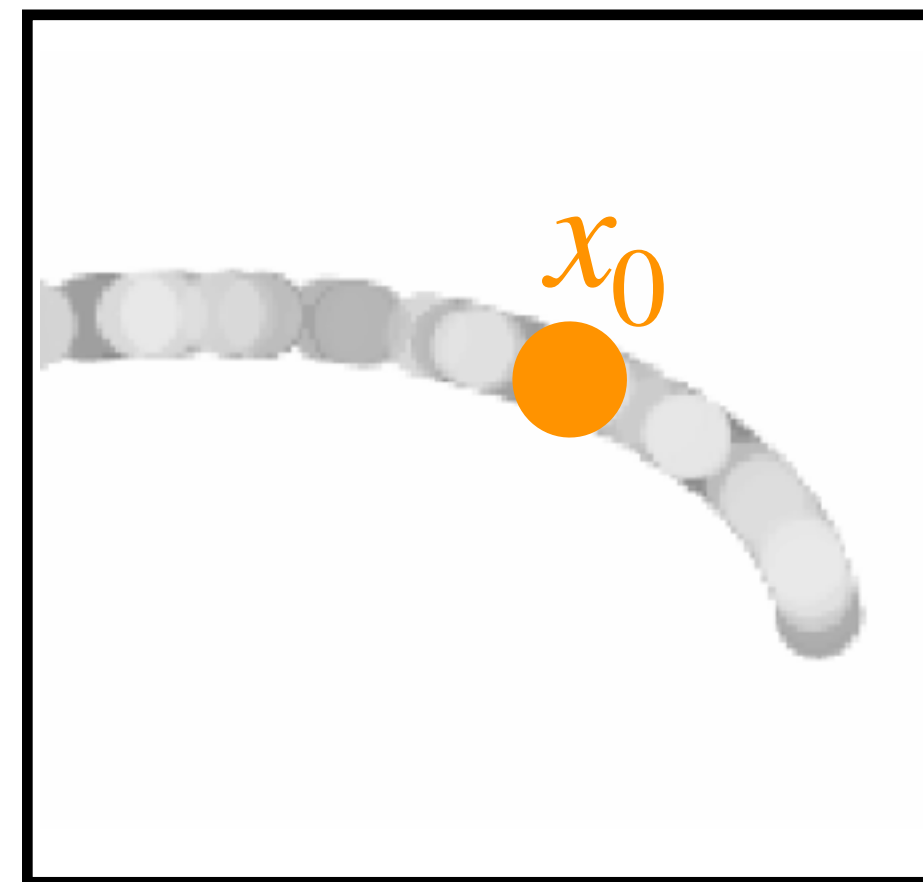
Joint $p(x_0, x_t)$:

It describes having one clean point x_0 and one noisy point x_t , occurring together



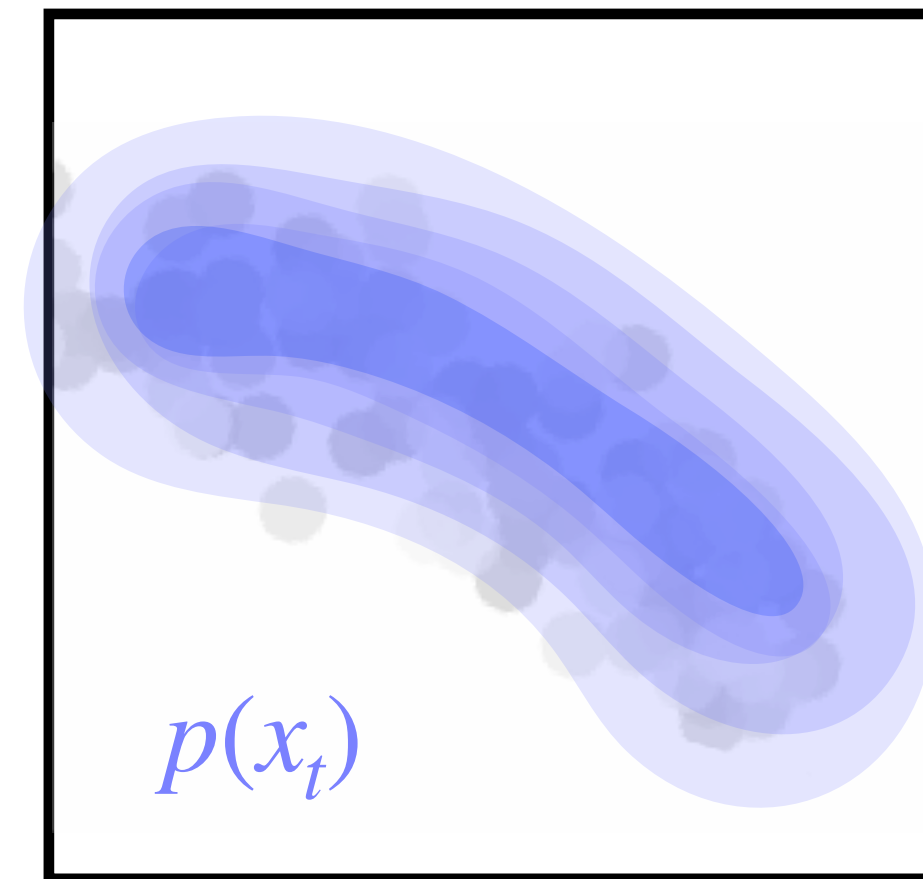
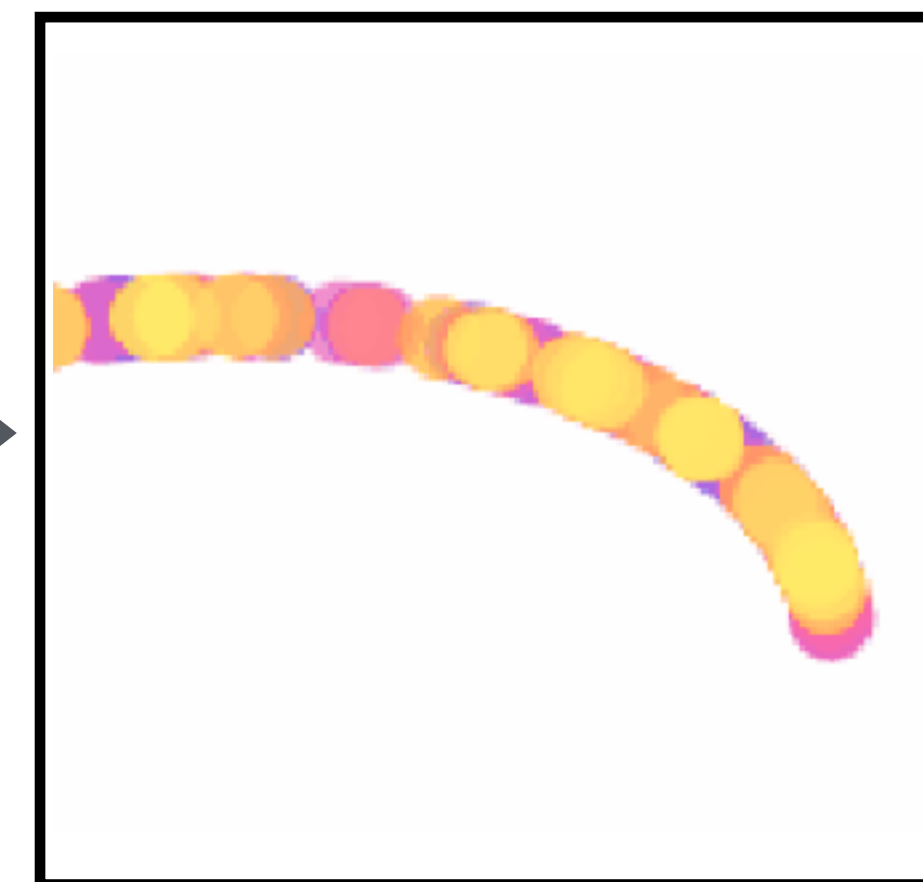
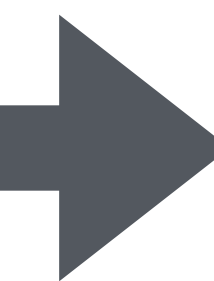
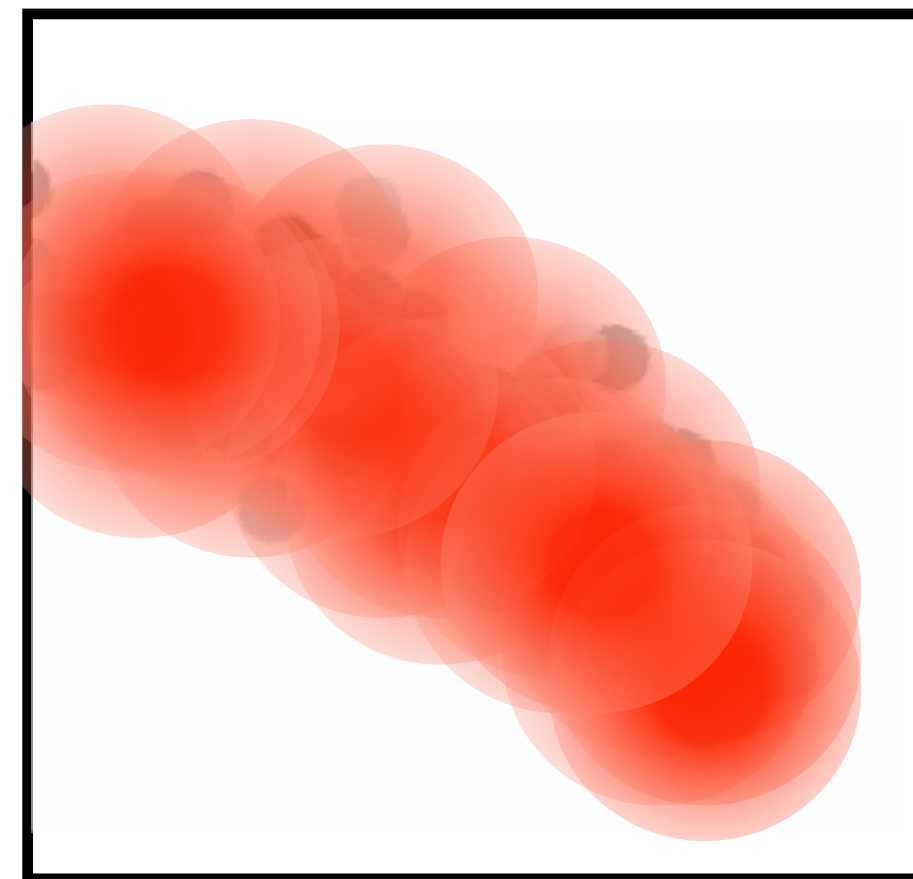
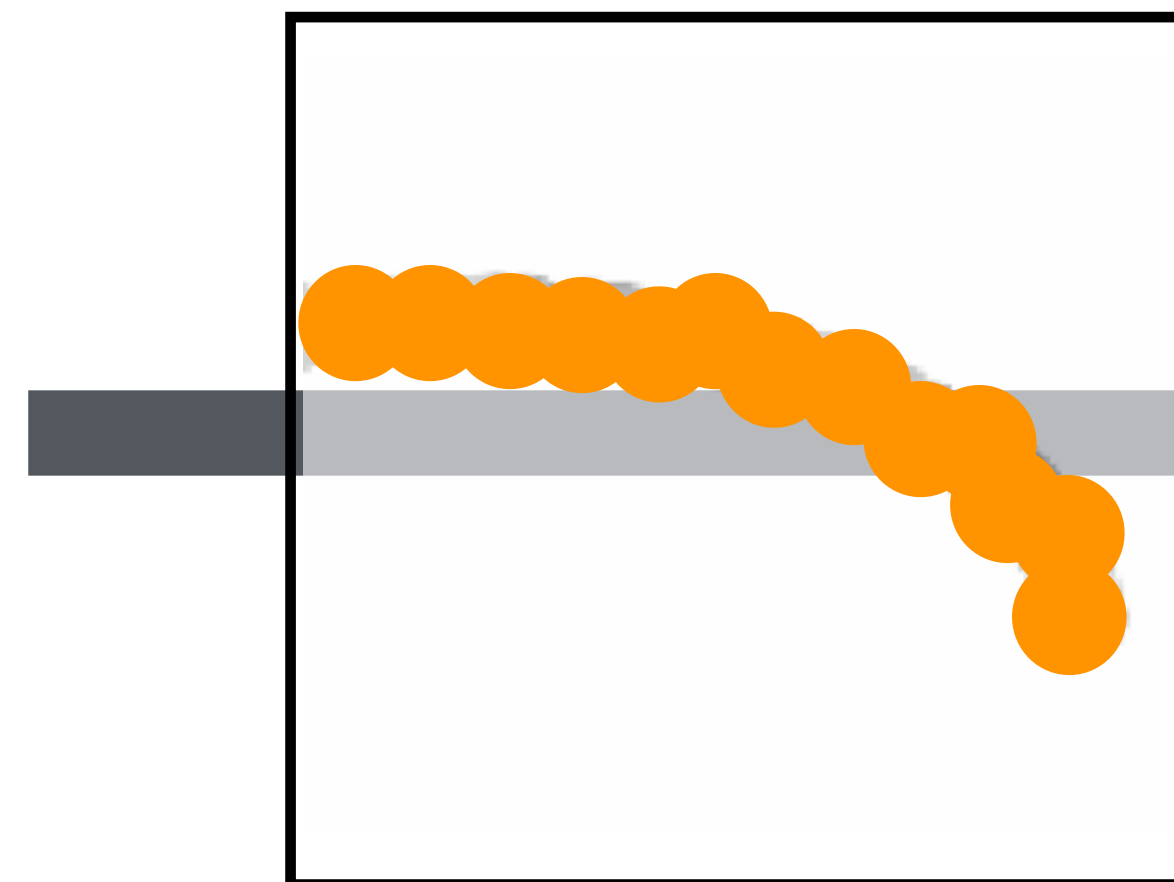
Conditional $p(x_t | x_0)$:

fix a specific clean point x_0 and ask, "what do its noisy versions could look like?" It's a gaussian blob



Marginal $p_t(x) = \int p(x_t | x_0) p_{data} dx_0$:

what does the noisy points x_t look like, not knowing where it came from?
So in marginal the source info is averaged away.



Clarifying why **conditional probability** is a Gaussian

In the discrete picture one **forward step** does only two simple things:
(for $\Delta t = 1$)

1) shrinks the it a little toward the origin (note $\sqrt{1 - \beta} \approx 1 - \frac{1}{2}\beta$)

2) Adds a little random Gaussian fuzz

$$x_1 = \sqrt{1 - \beta} x_0 + \sqrt{\beta} \epsilon$$

Or:

$$p(x_1 | x_0) = \mathcal{N} \left(\sqrt{1 - \beta} x_0, \beta I \right)$$

After t steps, one can prove that all the small shrink-and-fuzz steps combine into more shrinkage and bigger gaussian fuzz:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

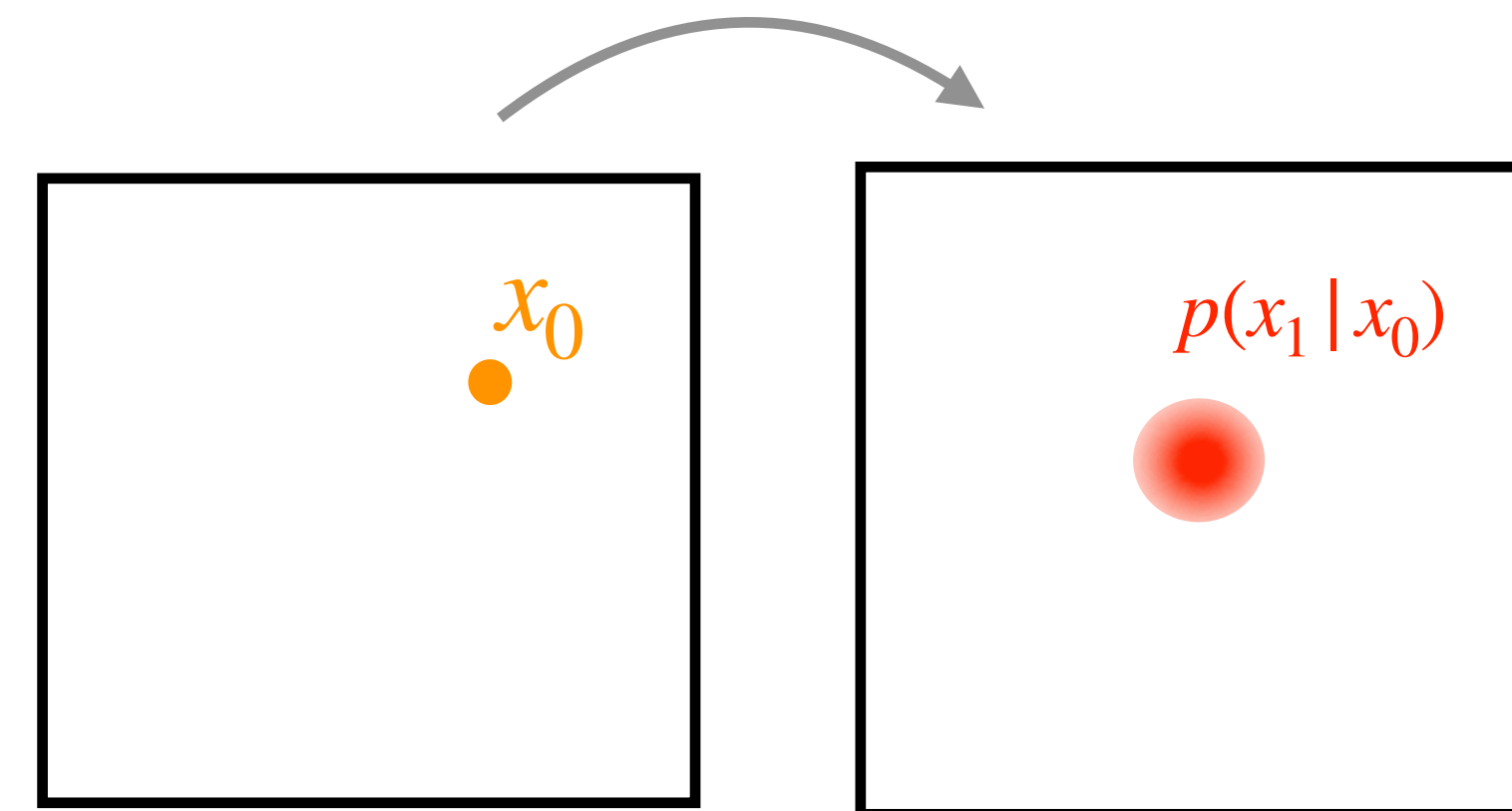
Or:

$$p(x_t | x_0) = \mathcal{N} \left(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I \right)$$

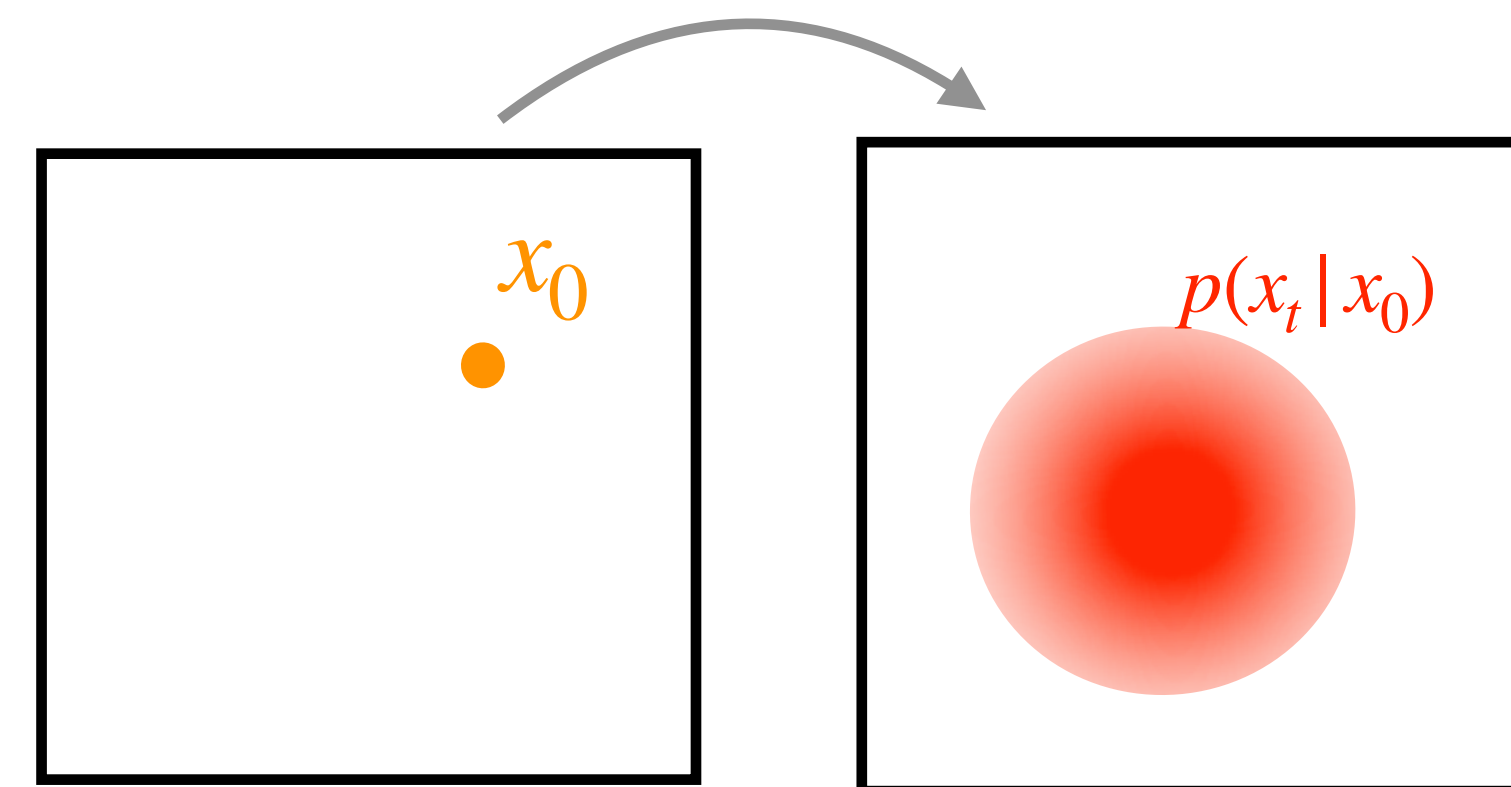
Where:

$$\alpha = 1 - \beta \quad , \quad \bar{\alpha}_t = \prod_{s=1}^t \alpha = \prod_{s=1}^t (1 - \beta)$$

One step later



t steps later



2) Reverse SDE process: a from noisy data to data

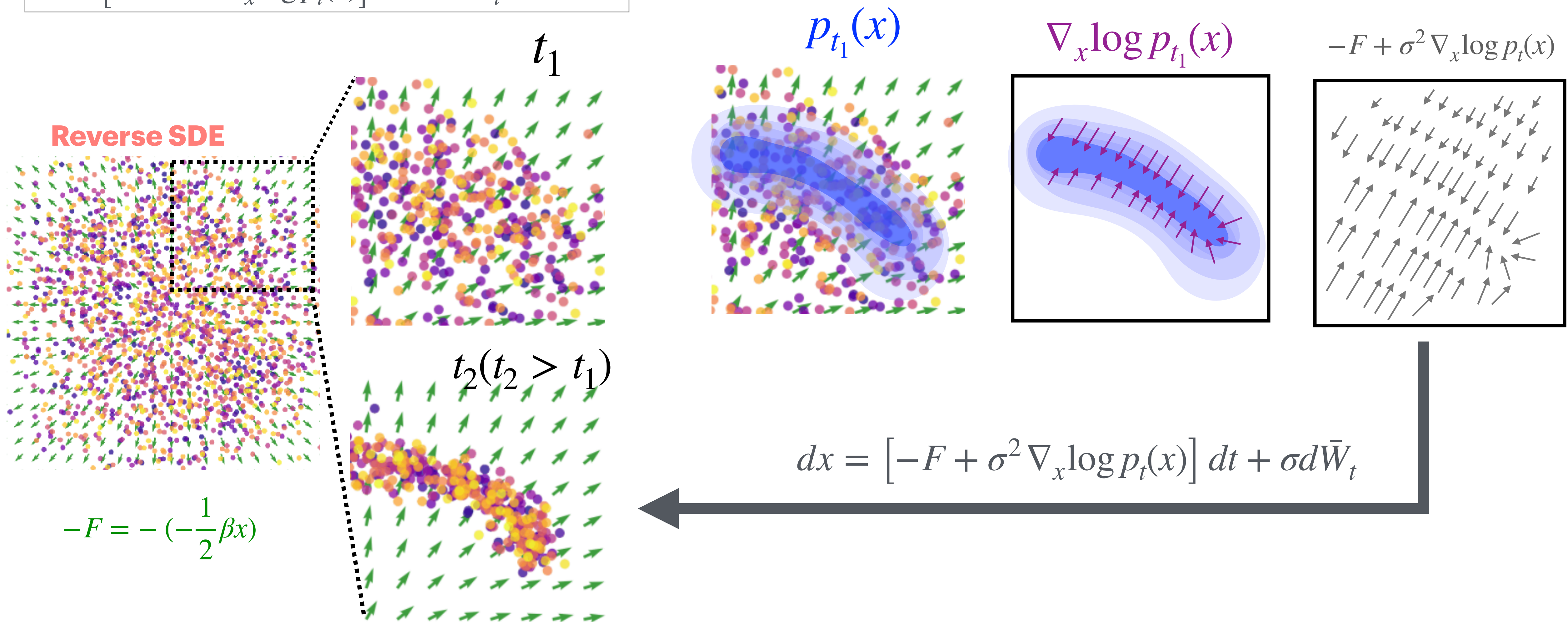
Reminder: reverse an SDE

- i) flip the force: $F(x, t) \rightarrow -F(x, t)$
- ii) We need to learn $\nabla_x \log p_t(x)$ (unknown)
- iii) Reverse SDE is:

$$dx = \left[-F + \sigma^2 \nabla_x \log p_t(x) \right] dt + \sigma d\bar{W}_t$$

Question: The force term of reverse process($-F$) is pointing towards out of origin. Why som points are getting closer to the origin in a coordinated way?

Answer:

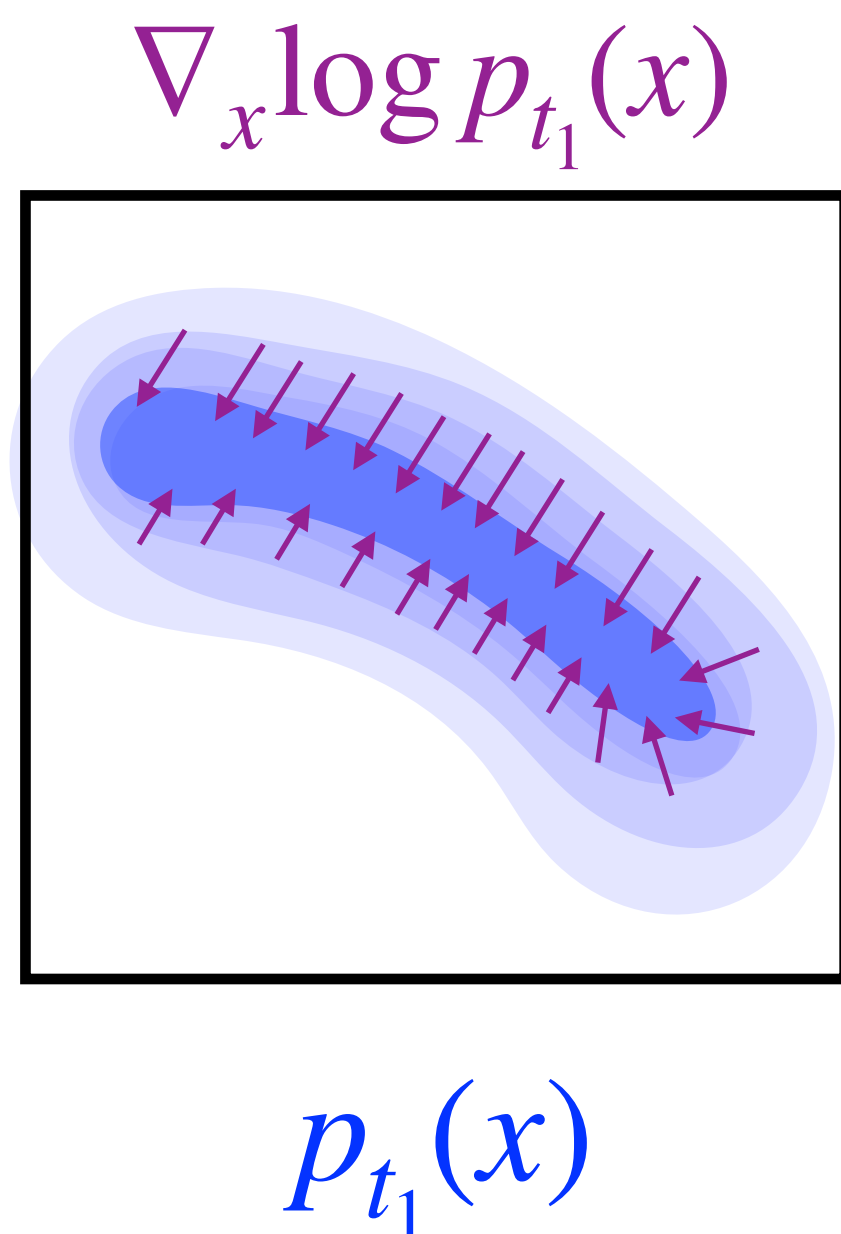


2) Reverse SDE process: We need to know $\nabla_x \log p_t(x)$

To reverse the diffusion process:

1) We need the $-F$: already known

2) We also need the density-correction field, at each time and space, $\nabla_x \log p_t(x)$. It is unknown. Let's give it a name:



$$\text{Score}(x, t) = \nabla_x \log p_t(x)$$

$p_t(x)$: Density of the noisy data cloud at time t at x

$\nabla_x \log p_t(x)$: Arrow pointing toward higher density at time t at x

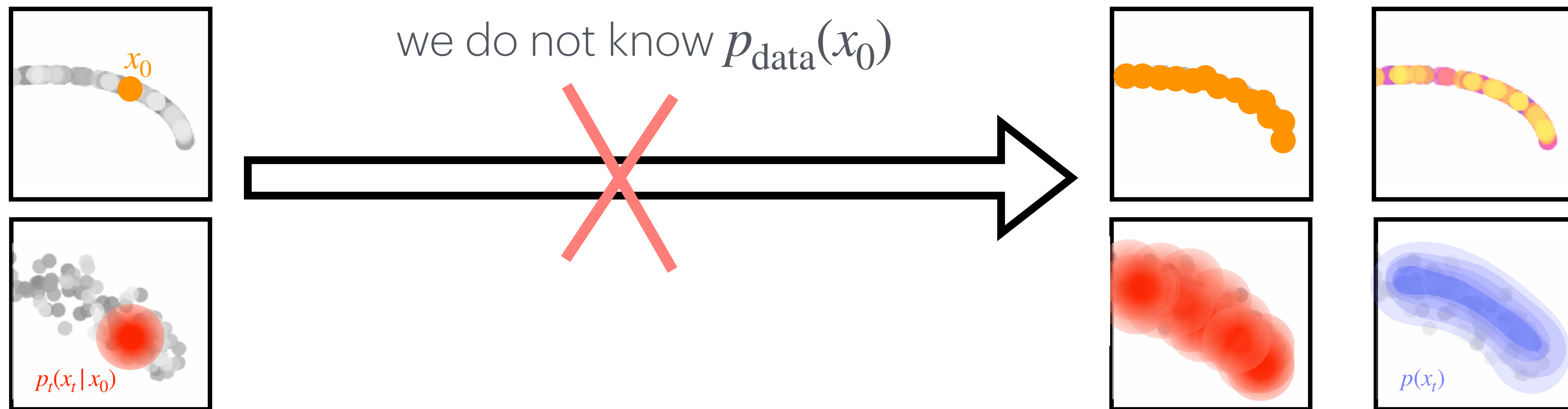
2) Reverse SDE process: Where to get $\nabla_x \log p_t(x)$?

The score field $\text{Score}(x, t) = \nabla_x \log p_t(x)$ is unknown to us. How can we get it?

Option 1 direct route) reconstruct the whole blue cloud ($p_t(x)$) first then get the gradient:

$$p_t(x) = \int p_t(x \mid x_0) p_{\text{data}}(x_0) dx_0$$

But we do not have $p_{\text{data}}(x_0)$ to begin with, we just have some samples, and the curse of dimensionality hits again

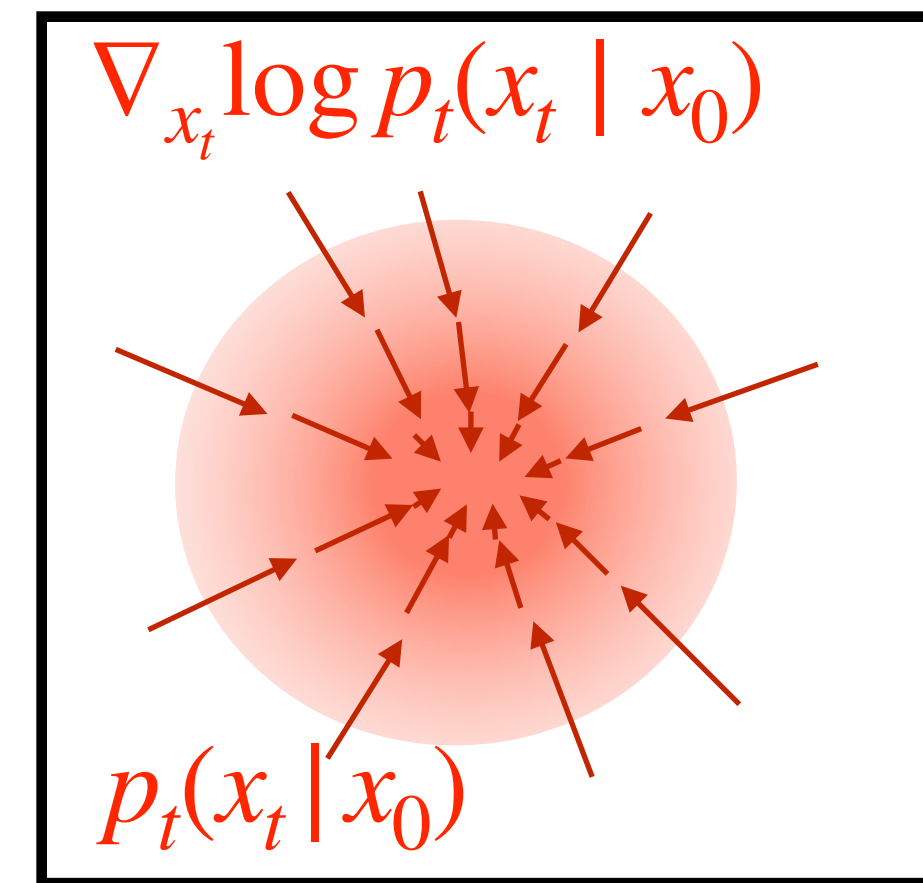
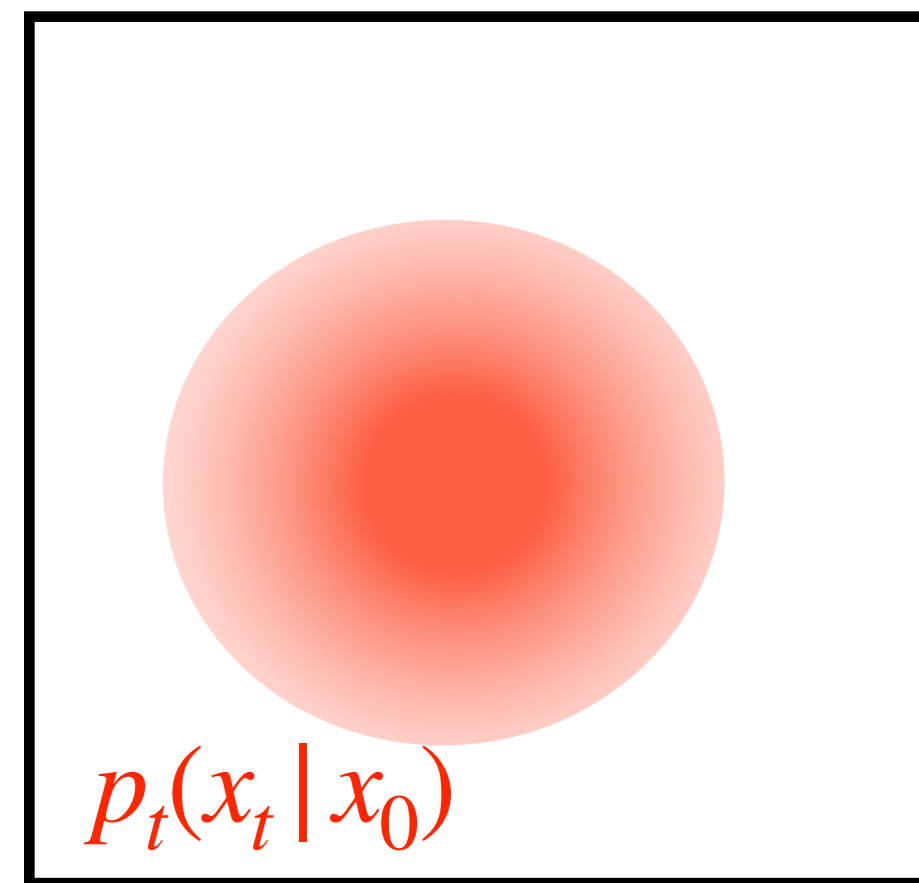
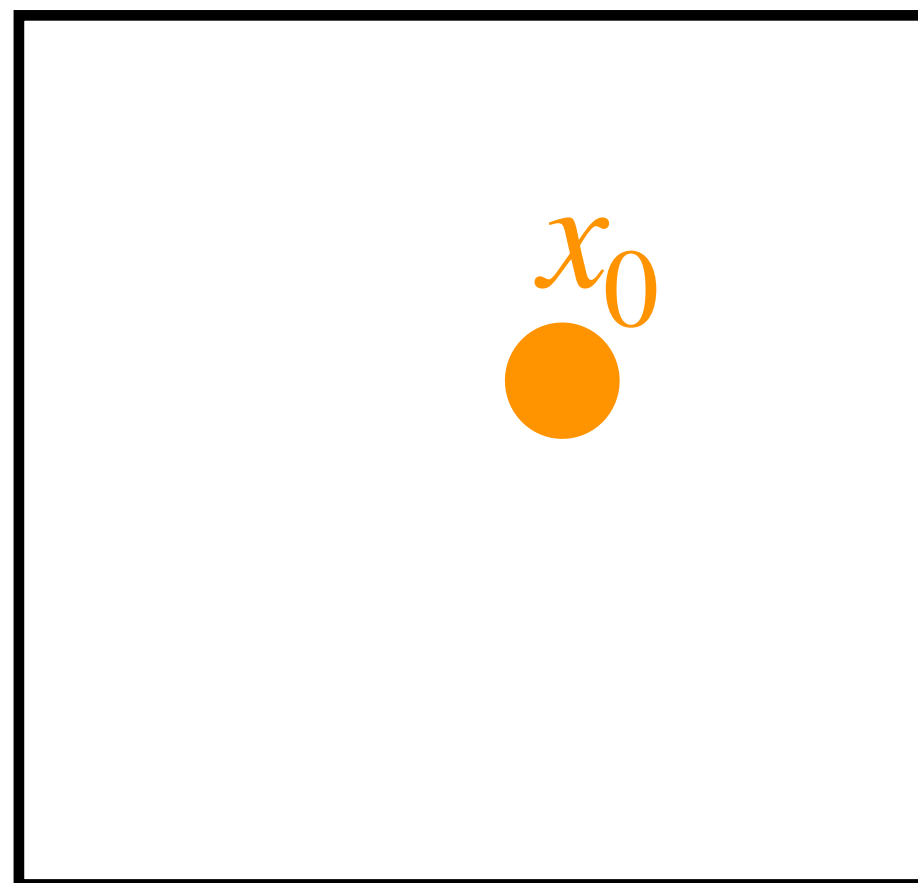


2) Reverse SDE process: Where to get $\nabla_x \log p_t(x)$?

Option 2 use what we do know, one Gaussian blob at a time for every training example (x_0)

we know how to convert it to x_t :

$$x_0 \rightarrow x_t \quad \text{via} \quad p_t(x_t | x_0) = \mathcal{N} \left(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I \right)$$



$\nabla_{x_t} \log p_t(x_t | x_0)$
is only for one blob

And we know how to calculate gradient of a gaussian blob easily:

$$p(x) = \mathcal{N}(x; \mu, \sigma^2 I) \quad \rightarrow \quad \nabla_x \log p(x) = -\frac{x - \mu}{\sigma^2}$$

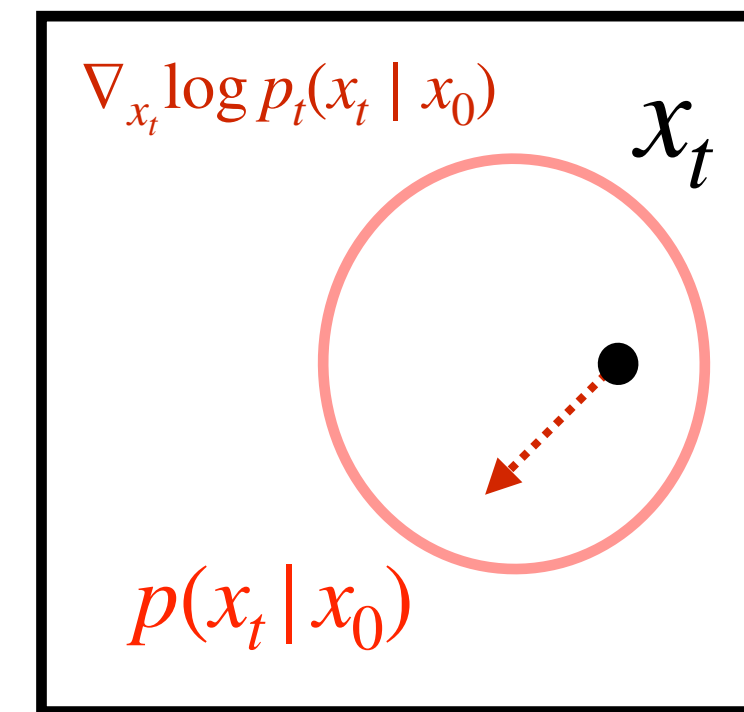
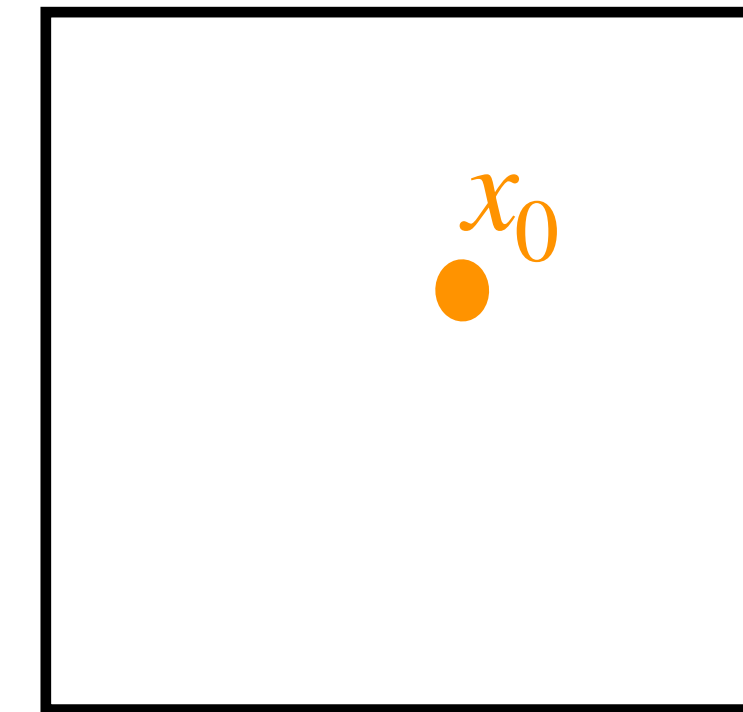
Therefore:

$$\nabla_{x_t} \log p_t(x_t | x_0) = \frac{\sqrt{\bar{\alpha}_t} x_0 - x_t}{1 - \bar{\alpha}_t}$$

2) Reverse SDE process: Where to get $\nabla_x \log p_t(x)$?

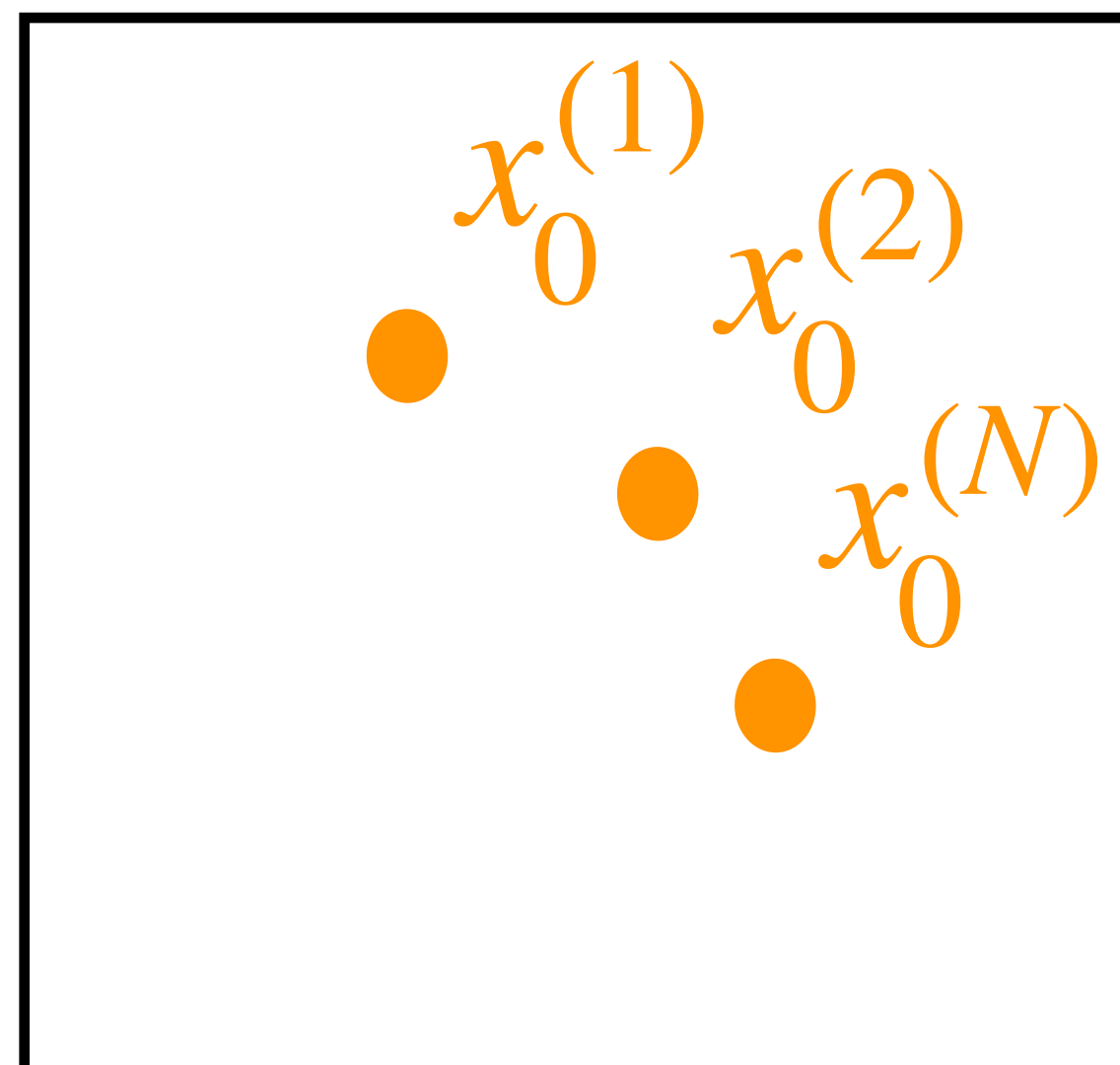
We can calculate the gradient related to x_0 at any arbitrary (x_t) :

$$\nabla_{x_t} \log p_t(x_t | x_0) = \frac{\sqrt{\bar{\alpha}_t} x_0 - x_t}{1 - \bar{\alpha}_t}$$



But what we want for **reverse SDE** is the score of the whole noisy cloud at any location (x_t) .

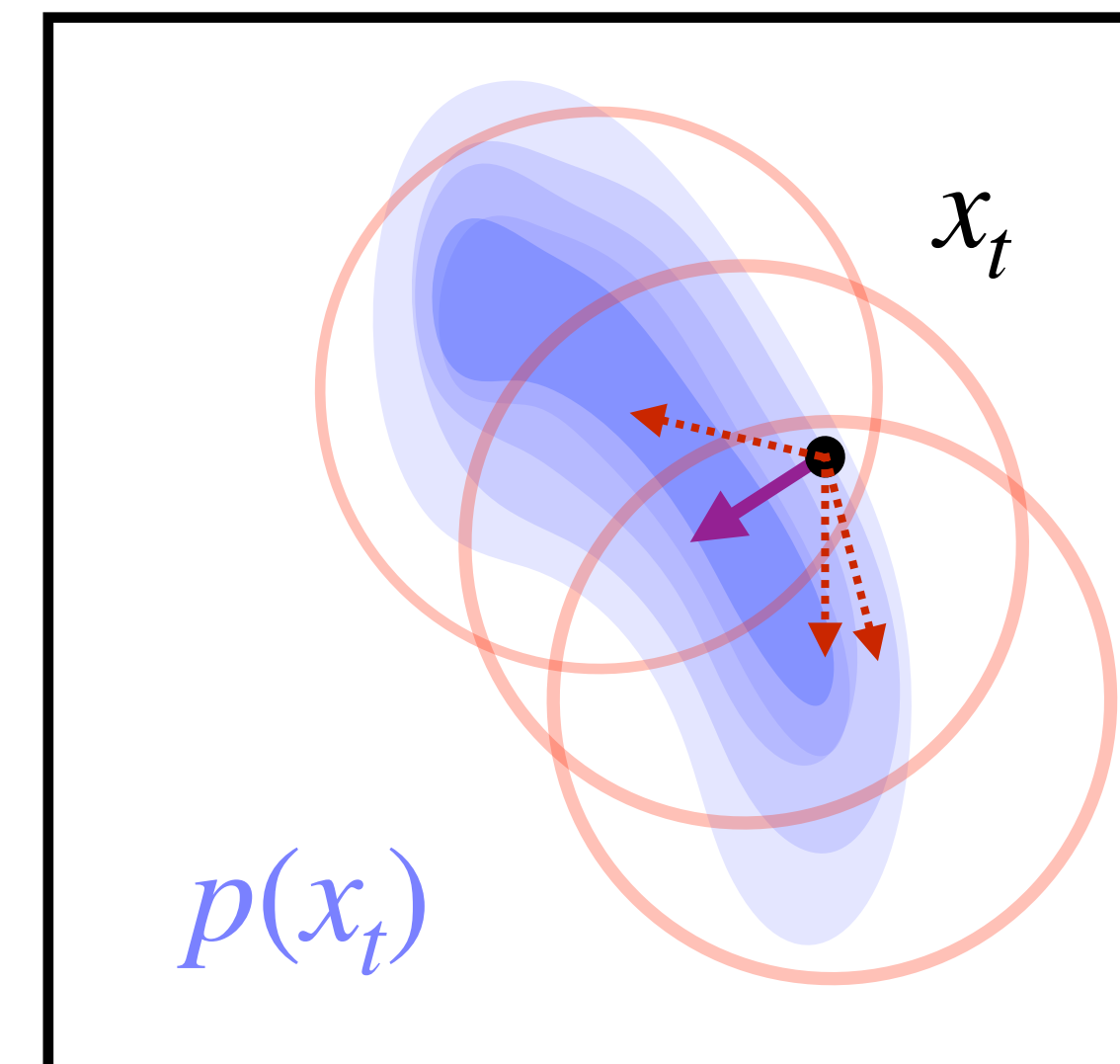
If we have $x_0^{(1)}, x_0^{(2)}, \dots, x_0^{(N)}$ clean points, **Score** is the weighed average of all blob contributions:



$$\text{Score}(x, t) = \nabla_x \log p_t(x) \approx \sum_{i=1}^N w_i(x_t) \nabla_{x_t} \log p_t(x_t | x_0^{(i)})$$

$$\text{Where: } w_i(x_t) = p_t(x_0^{(i)} | x_t)$$

score of full cloud = weighted average of individual blob scores



2) Reverse SDE process: Where to get $\nabla_x \log p_t(x)$? Train a neural network to output it

We can train a Neural network (S_θ) that learns the score:

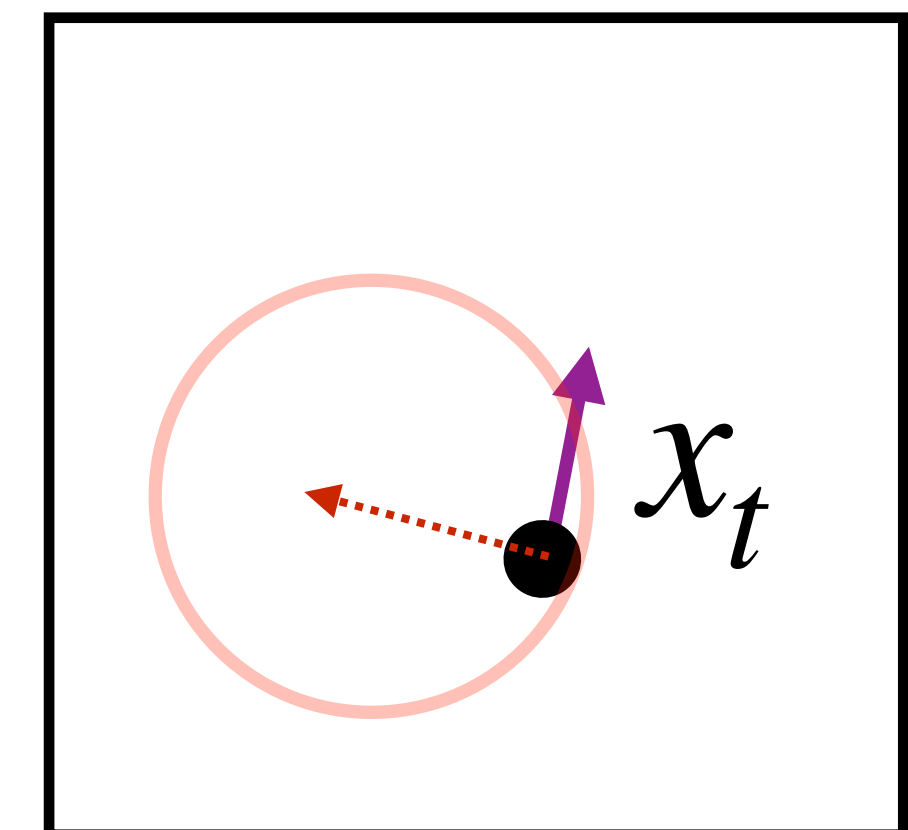
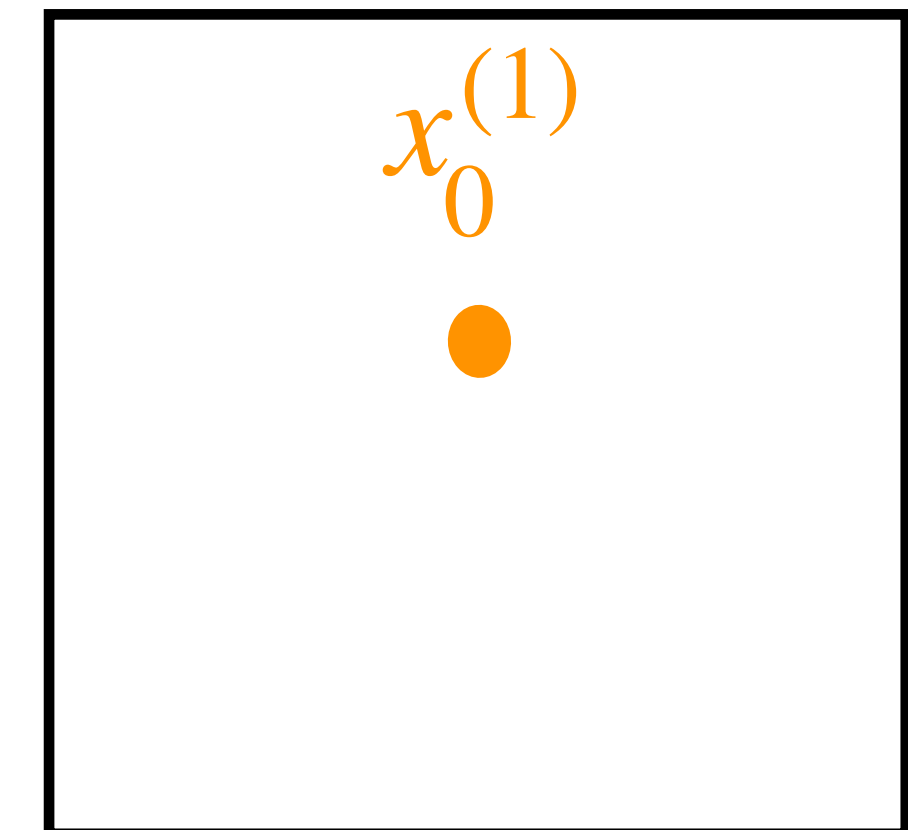
$$s_\theta(x, t) \approx \nabla_x \log p_t(x)$$

toy training loop:

1. Sample one clean data point: $x_0 \sim p_{\text{data}}$
2. Choose a random noise level: $t \sim \text{Uniform}\{1, \dots, T\} \rightarrow \bar{\alpha}_t$,
3. Sample Gaussian noise: $\epsilon \sim \mathcal{N}(0, I)$
4. Create one noisy point: $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$
5. Calculate the arrow of the blob that produced it:

$$a = \nabla_{x_t} \log p_t(x_t | x_0) = \frac{\sqrt{\bar{\alpha}_t} x_0 - x_t}{1 - \bar{\alpha}_t}$$

6. Ask the neural network to predict that arrow: $\hat{a} = s_\theta(x_t, t)$
7. Calculate the loss for that single point: $\ell(\theta) = \| \hat{a} - a \|^2$
8. Back propagate



2) Reverse SDE process: Where to get $\nabla_x \log p_t(x)$? Train a neural network to output it

We can train a Neural network (S_θ) that learns the score:

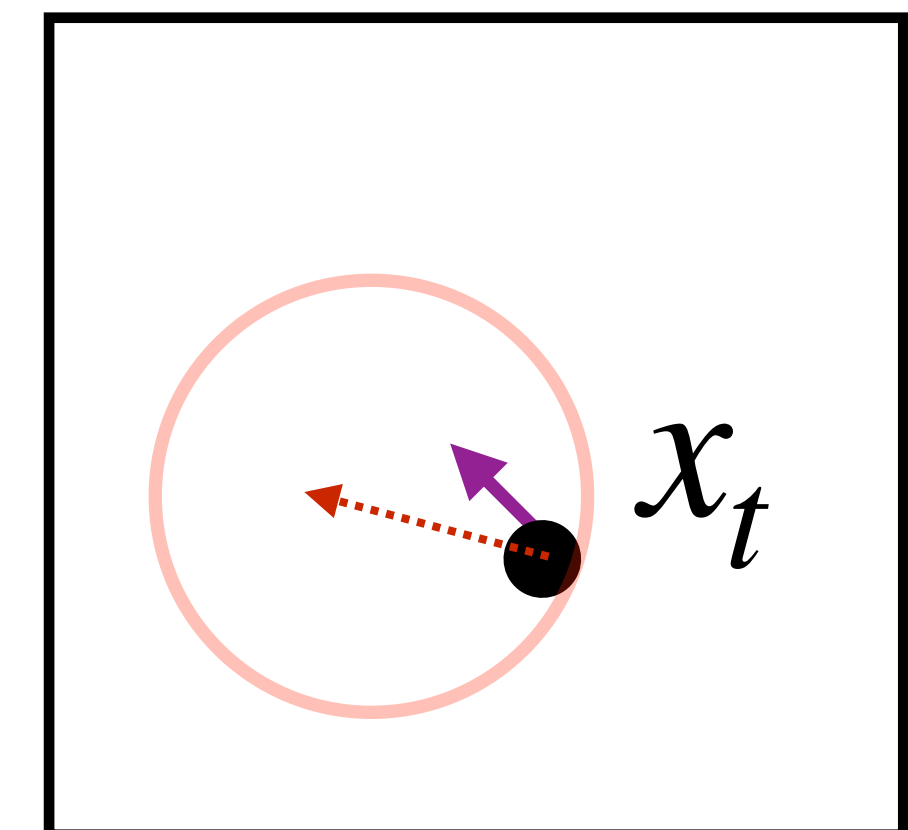
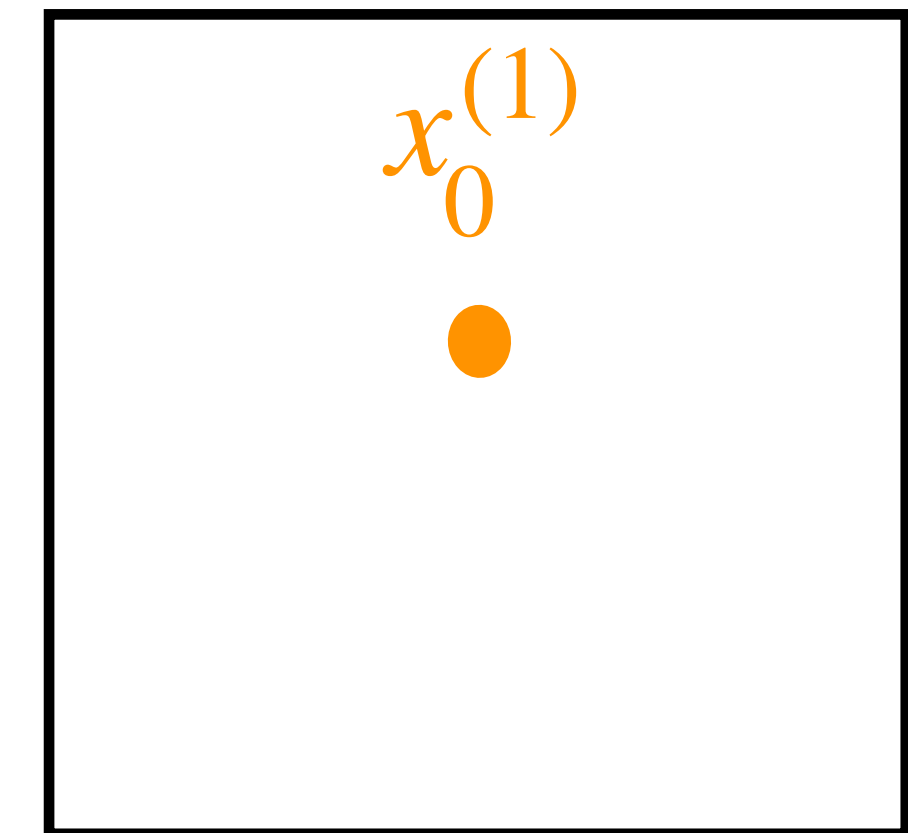
$$s_\theta(x, t) \approx \nabla_x \log p_t(x)$$

toy training loop:

1. Sample one clean data point: $x_0 \sim p_{\text{data}}$
2. Choose a random noise level: $t \sim \text{Uniform}\{1, \dots, T\} \rightarrow \bar{\alpha}_t$,
3. Sample Gaussian noise: $\epsilon \sim \mathcal{N}(0, I)$
4. Create one noisy point: $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$
5. Calculate the arrow of the blob that produced it:

$$a = \nabla_{x_t} \log p_t(x_t | x_0) = \frac{\sqrt{\bar{\alpha}_t} x_0 - x_t}{1 - \bar{\alpha}_t}$$

6. Ask the neural network to predict that arrow: $\hat{a} = s_\theta(x_t, t)$
7. Calculate the loss for that single point: $\ell(\theta) = \| \hat{a} - a \|^2$
8. Back propagate



2) Reverse SDE process: Where to get $\nabla_x \log p_t(x)$? Train a neural network to output it

We can train a Neural network ($\mathcal{S}_\theta$) that learns the score:

$$s_\theta(x, t) \approx \nabla_x \log p_t(x)$$

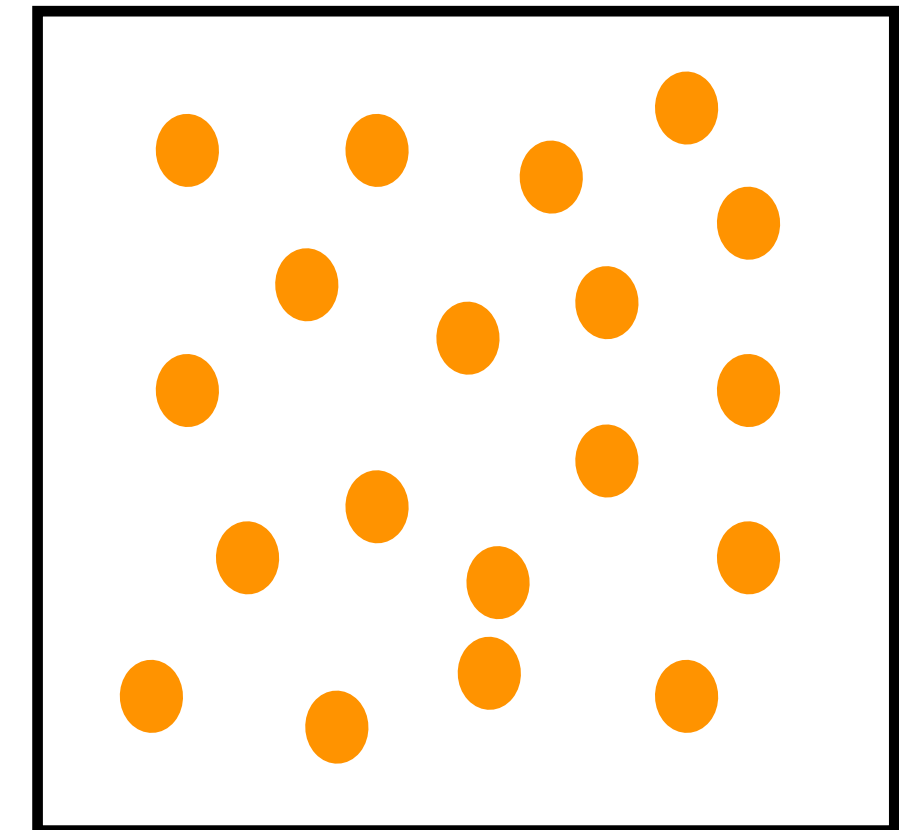
toy training loop:

1. Sample one clean data point: $x_0 \sim p_{\text{data}}$
2. Choose a random noise level: $t \sim \text{Uniform}\{1, \dots, T\} \rightarrow \bar{\alpha}_t$
3. Sample Gaussian noise: $\epsilon \sim \mathcal{N}(0, I)$
4. Create one noisy point: $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$
5. Calculate the arrow of the blob that produced it:

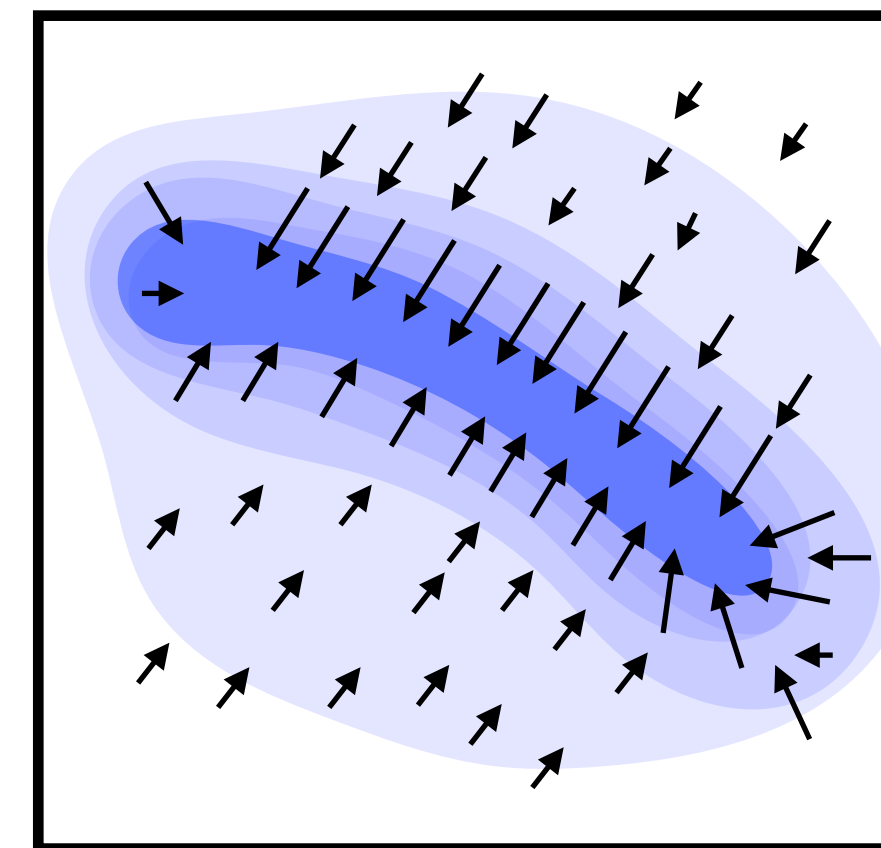
$$a = \nabla_{x_t} \log p_t(x_t | x_0) = \frac{\sqrt{\bar{\alpha}_t} x_0 - x_t}{1 - \bar{\alpha}_t}$$

6. Ask the neural network to predict that arrow: $\hat{a} = s_\theta(x_t, t)$
7. Calculate the loss for that single point: $\ell(\theta) = \| \hat{a} - a \|^2$
8. Back propagate

Repeat with many clean points, noise levels, and noise samples

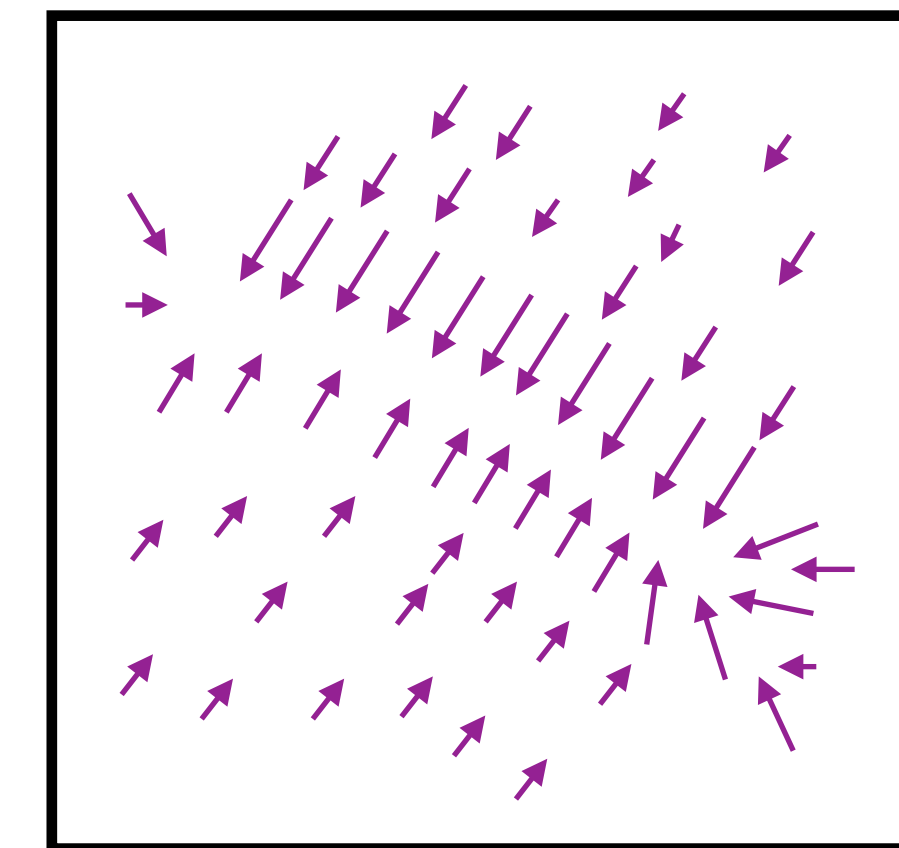


$\nabla_x \log p_t(x)$



$\approx$

$s_\theta(x_t, t)$



This is the implementation of Training a Diffusion model

Song, Yang, and Stefano Ermon. "Generative modeling by estimating gradients of the data distribution." *Advances in neural information processing systems* 32 (2019)

Song, Yang, et al. "Score-based generative modeling through stochastic differential equations." *arXiv preprint arXiv:2011.13456* (2020).

Training loop: one mini-batch

0. Draw a mini-batch of B independent examples: $i = 1, \dots, B$

1. Sample one clean data point: $x_0^{(i)} \sim p_{\text{data}}, \quad i = 1, \dots, B$

2. Choose a random noise level: $t^{(i)} \sim \text{Uniform}\{1, \dots, T\} \rightarrow \bar{\alpha}_{t^{(i)}}$,

3. Sample Gaussian noise: $\epsilon^{(i)} \sim \mathcal{N}(0, I)$

4. Create one noisy point: $x_{t^{(i)}}^{(i)} = \sqrt{\bar{\alpha}_{t^{(i)}}} x_0^{(i)} + \sqrt{1 - \bar{\alpha}_{t^{(i)}}} \epsilon^{(i)}$

5. Calculate the arrow of the blob that produced it:

$$a^{(i)} = \nabla_{x_{t^{(i)}}^{(i)}} \log p_{t^{(i)}} \left(x_{t^{(i)}}^{(i)} \mid x_0^{(i)} \right) = \frac{\sqrt{\bar{\alpha}_{t^{(i)}}} x_0^{(i)} - x_{t^{(i)}}^{(i)}}{1 - \bar{\alpha}_{t^{(i)}}}$$

6. Ask the neural network to predict that arrow: $\hat{a}^{(i)} = s_{\theta} \left(x_{t^{(i)}}^{(i)}, t^{(i)} \right)$

7. Calculate the loss for each single point: $\ell^{(i)}(\theta) = \left\| \hat{a}^{(i)} - a^{(i)} \right\|^2$

8. Average the losses across the batch: $\mathcal{L}_{\text{batch}}(\theta) = \frac{1}{B} \sum_{i=1}^B \ell^{(i)}(\theta)$

9. Back propagate, update θ .

Generating new data: run the learned reverse SDE from noise to data

Generating New data

0. Training gave us: $s_\theta(x, t) \approx \nabla_x \log p_t(x)$

1. We start draw a random noisy point: $x_T \sim \mathcal{N}(0, I)$

2. We know the reverse SDE:

$$dx = \left[-F(x, t) + \sigma(t)^2 \nabla_x \log p_t(x) \right] d\tau + \sigma(t) d\bar{W}_\tau$$

we have $F = -\frac{1}{2}\beta(t)x$, $s_\theta(x, t) \approx \nabla_x \log p_t(x)$, $\sigma(t)^2 = \beta(t)$:

$$dx = \left[\frac{1}{2}\beta(t)x + \beta(t)s_\theta(x, t) \right] d\tau + \sqrt{\beta(t)} d\bar{W}_\tau$$

3. To solve it, we use Euler-Maruyama steps:

$$x_{t-\Delta t} = x_t + \left[\frac{1}{2}\beta(t)x_t + \beta(t)s_\theta(x_t, t) \right] \Delta t + \sqrt{\beta(t)\Delta t} \epsilon_t$$

Where: $\Delta t = \frac{T}{N}$, $t = T, T - \Delta t, \dots, \Delta t, 0$,

“At each reverse step, the neural network looks at the current noisy point and tells us which direction leads toward more data-like regions. We take a small step in that direction”

Recipe Card: **Diffusion Models , Training**

Type of Learning: **Self-supervised**

Data set: **Any data**

Neural network forward process Data

- 1- Load libraries: Basics + PyTorch
- 2-Load your Dataset: $\mathcal{D} = \{x_0^{(i)}\}_{i=1}^N$
 - Each $x_0^{(i)}$ is one clean data example.
- 3-Now set up the DataLoader for training/validation/testing:
 - Create standard train / validation / test splits and mini-batches.
- 4- Define the fixed forward process for one point $x_0^{(i)}$.
 - 4.1-Choose: T = number of noise levels
 - 4.2-Define: $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s = \prod_{s=1}^t (1 - \beta_s)$
 - 4.4-For one point, $x_0^{(i)}$ where $i = 1, \dots, N$
 - 4.5-For $x_0^{(i)}$, choose a random noise level: $t \sim \text{Uniform}\{1, \dots, T\}$
 - 4.6-Sample a Gaussian number from: $\epsilon \sim \mathcal{N}(0, I)$
 - 4.7- Create one noisy version of $x_0^{(i)}$: $x_t^{(i)} = \sqrt{\bar{\alpha}_t} x_0^{(i)} + \sqrt{1 - \bar{\alpha}_t} \epsilon$
- 5- Build the score neural network
 - The neural network receives: (x_t, t)
 - It outputs a vector with the same shape as x_t : $s_\theta(x_t, t)$
 - The goal is: $s_\theta(x_t, t) \approx \nabla_{x_t} \log p_t(x_t)$
 - Architecture depends on the data structure.

Training

- 6- Calculate the score for mini batch $\{x_0^{(b)}\}_{b=1}^B$
 - For one mini-batch $x_0^{(i)}$, $b = 1, \dots, B$.
 - For each $x_0^{(b)}$ the exact score is:

$$a^{(b)} = \nabla_{x_{t^{(b)}}} \log p_{t^{(b)}} \left(x_{t^{(b)}}^{(b)} \mid x_0^{(b)} \right) = \frac{\sqrt{\bar{\alpha}_{t^{(b)}}} x_0^{(b)} - x_{t^{(b)}}^{(b)}}{1 - \bar{\alpha}_{t^{(b)}}}$$
- 7- Define the score-matching loss:
 - Predict the score via NN is: $\hat{a}^{(b)} = s_\theta \left(x_{t^{(b)}}^{(b)}, t^{(b)} \right)$
 - For each $x_0^{(b)}$ the stable loss is: $\ell^{(b)}(\theta) = \left(1 - \bar{\alpha}_{t^{(b)}} \right) \left\| \hat{a}^{(b)} - a^{(b)} \right\|^2$
 - Loss for a mini-batch: $\mathcal{L}_{\text{batch}}(\theta) = \frac{1}{B} \sum_{b=1}^B \ell^{(b)}(\theta)$
- 8- Training loop for each point in mini-batch $\{x_0^{(b)}\}_{b=1}^B$
 - 8.1-Draw $\{x_0^{(b)}\}_{b=1}^B$, $\{t^{(b)}\}_{b=1}^B$ and $\{\epsilon^{(b)}\}_{b=1}^B$
 - 8.2-compute $a^{(b)}$ and $\hat{a}^{(b)}$
 - 8.3-compute $\mathcal{L}_{\text{batch}}(\theta) = \frac{1}{B} \sum_{b=1}^B \ell^{(b)}(\theta)$
 - 8.4-Backpropagate $\nabla_\theta \mathcal{L}_{\text{batch}}(\theta)$
 - 8.5- Repeat for many mini-batches and epochs.

9. Diagnostics after training

Recipe Card: **Diffusion Models , Generating New Samples**

Type of Learning: **generative modeling**

Data set: **Any data**

10- Load the trained diffusion model:

- Load score model: $s_{\theta}(x, t) \approx \nabla_x \log p_t(x)$
- Load the same forward noise schedule: $\beta(t)$
- Choose the number of reverse solver steps: K
- This sets: $\Delta t = \frac{T}{K}$

11- Start from pure noise as initial point:

- Pick a random initial point: $x_T^{new} \sim \mathcal{N}(0, I)$

12- Run the learned reverse SDE:

12.1-For $t = T, T - \Delta t, \dots, \Delta t$. We start from $t = T$

12.2-sample the noise of SDE as $\epsilon_t \sim \mathcal{N}(0, I)$

12.3-Get the predicted score: $\hat{s}_t = s_{\theta}(x_t^{new}, t)$

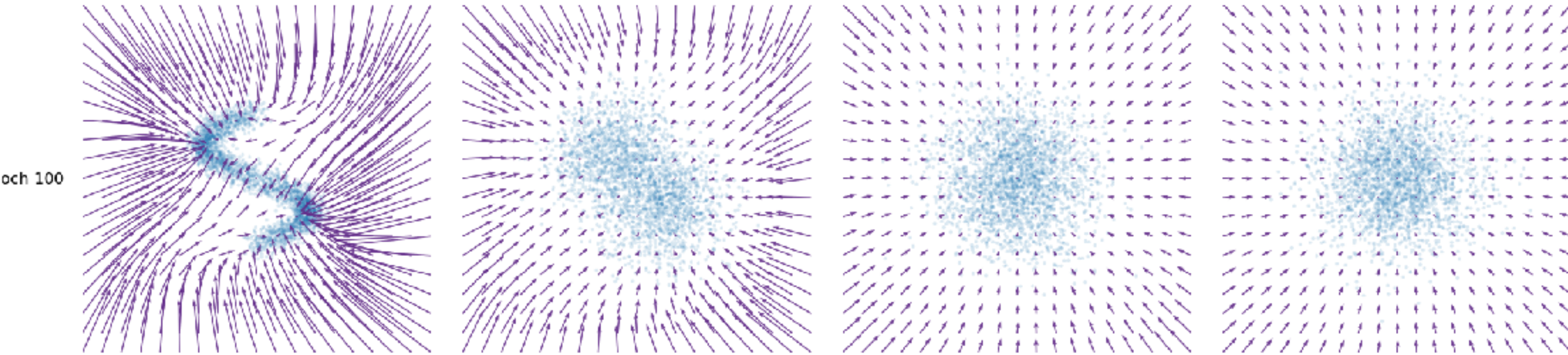
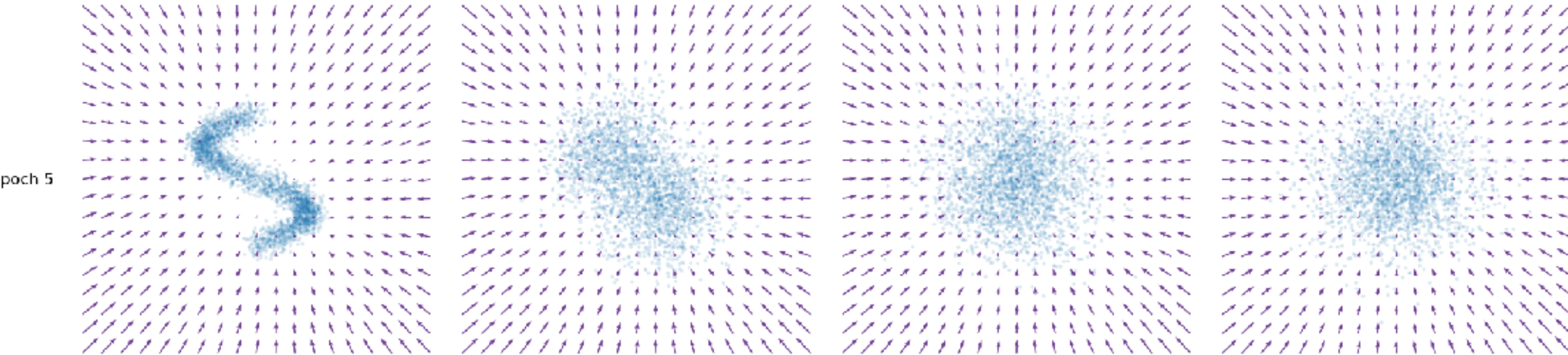
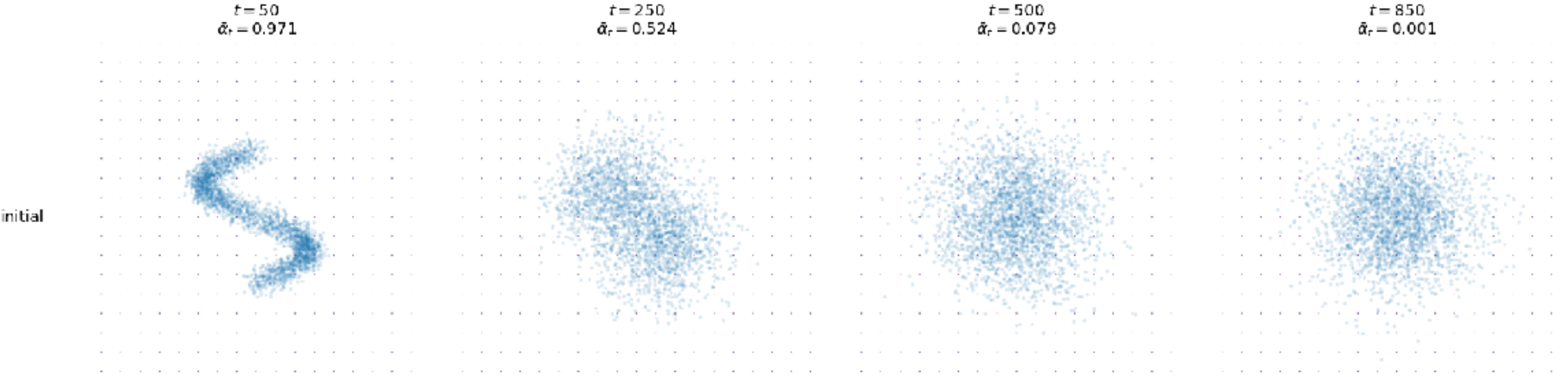
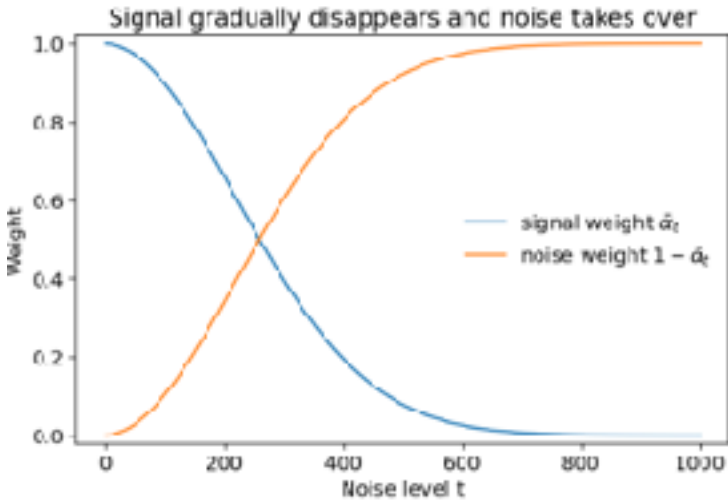
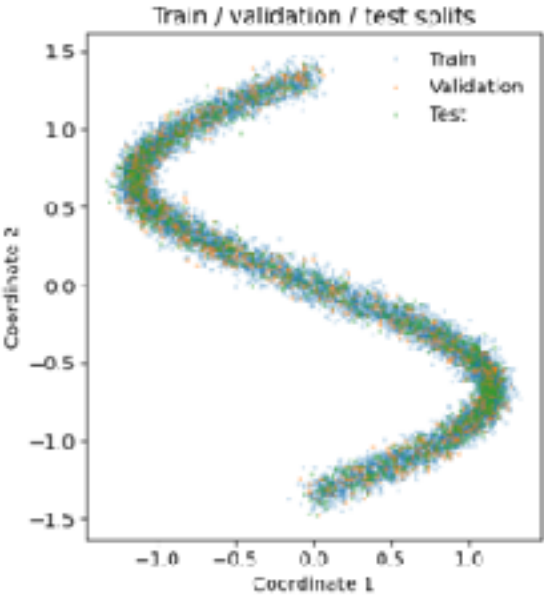
12.4-Then take one reverse Euler-Maruyama step:
$$x_{t-\Delta t}^{new} = x_t^{new} + \left[\frac{1}{2} \beta(t) x_t^{new} + \beta(t) s_{\theta}(x_t^{new}, t) \right] \Delta t + \sqrt{\beta(t) \Delta t} \epsilon_t$$

-Repeat (12.1-12.4) until getting to $t = 0$, and get $x_{t=0}^{new}$

-Note: in last step we can use $\epsilon_{\Delta t=0} = 0$

Everything everywhere all at once

$$s_{\theta}(x, t)$$





We will draw examples in two dimensions, but high-dimensional spaces behave very differently. For a 100-dimensional standard Gaussian, over 84% of samples have radius between 9 and 11, while fewer than 0.2% have radius below 8.

	$\mathcal{N}(0, I_2)$	$\mathcal{N}(0, I_{100})$
$\Pr(9 \leq r \leq 11)$	≈ 0	≈ 0.8428
$\Pr(8 \leq r \leq 12)$	≈ 0	≈ 0.9954
$\Pr(r < 8)$	≈ 1	≈ 0.00193

Appendix

We know: $p_t(x_t) = \int p_t(x_t \mid x_0) p_{\text{data}}(x_0) dx_0$

Then: $\nabla_{x_t} \log p_t(x_t) = \frac{\nabla_{x_t} p_t(x_t)}{p_t(x_t)} = \frac{\int \nabla_{x_t} p_t(x_t \mid x_0) p_{\text{data}}(x_0) dx_0}{p_t(x_t)}$

We know that $\nabla p = p \nabla \log p$

$$\nabla_{x_t} \log p_t(x_t) = \int \underbrace{\frac{p_t(x_t \mid x_0) p_{\text{data}}(x_0)}{p_t(x_t)}}_{\text{how likely was } x_0 \text{ the source of this } x_t} \nabla_{x_t} \log p_t(x_t \mid x_0) dx_0$$

The bracket is $p_t(x_0 \mid x_t)$

So we have: $\nabla_{x_t} \log p_t(x_t) = \mathbb{E}_{x_0 \sim p_t(x_0 \mid x_t)} \left[\nabla_{x_t} \log p_t(x_t \mid x_0) \right]$

Read more: Some good references

Great blog posts: sander.ai/2023/07/20/perspectives.html
<https://yang-song.net/blog/2021/score/>

Papers:

Zheng, Candi, and Yuan Lan. "Rethinking the Diffusion Model from a Langevin Perspective." arXiv preprint arXiv:2604.10465 (2026).

Holderrieth, Peter, and Ezra Erives. "An introduction to flow matching and diffusion models." arXiv preprint arXiv:2506.02070 (2025).

Lai, Chieh-Hsin, et al. "The principles of diffusion models." arXiv preprint arXiv:2510.21890 (2025).

Song, Yang, et al. "Score-based generative modeling through stochastic differential equations." arXiv preprint arXiv:2011.13456 (2020).

Song, Jiaming, Chenlin Meng, and Stefano Ermon. "Denoising diffusion implicit models." arXiv preprint arXiv:2010.02502 (2020).

Yang, Ling, et al. "Diffusion models: A comprehensive survey of methods and applications." ACM computing surveys 56.4 (2023): 1-39.